COMP H2704 – Web Application Security
TU757 / TU863
CA3 Project (50%)

Technical Report

Members:

Nana Sarpong

Ibrahim Sow

Lecturer: Dr Vanessa Ayala-Rivera

Fill out one of these submission coversheet for each team member
School of Informatics & Cybersecurity - Assessment Submission Coversheet
Student Information
,

Student Name: Nana Sarpong

Student Name: Ibrahima Sow

Module Information
Module Title:
Web Application Security
Module Code:
COMP H2704

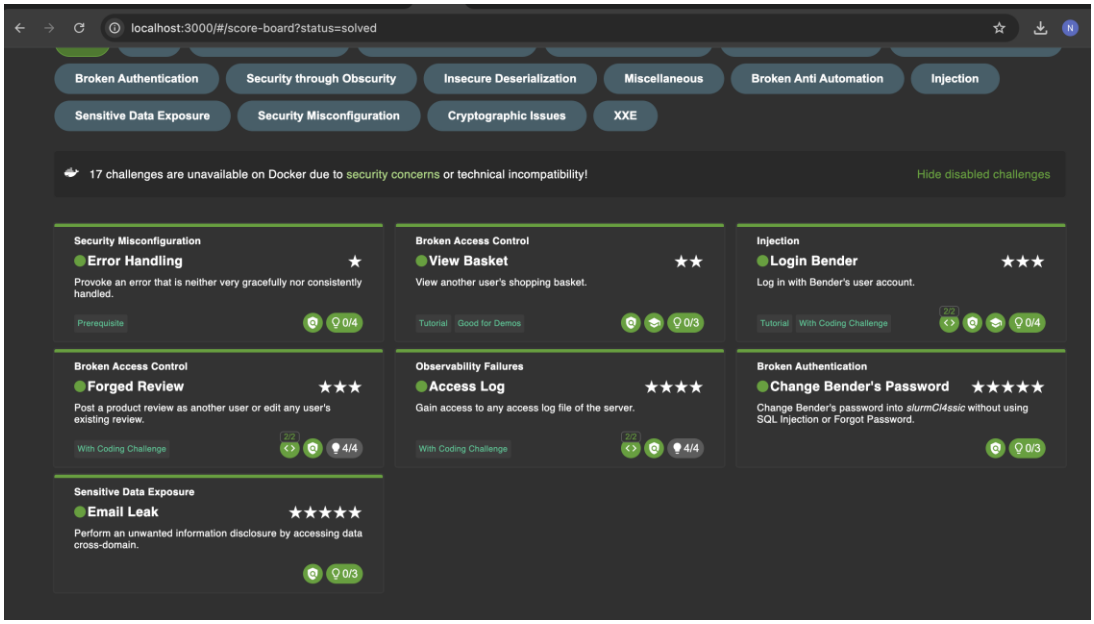
Declaration
☐ I confirm that I have read and understood the University's Academic Integrity Policy.
https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/
☐ I have completed the TU Dublin AI Driving License theory test.
https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

☐ I declare that this assessment submission is my own work.
☐ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature: nana sarpong, Ibrahim sow

Date: 27/04/26

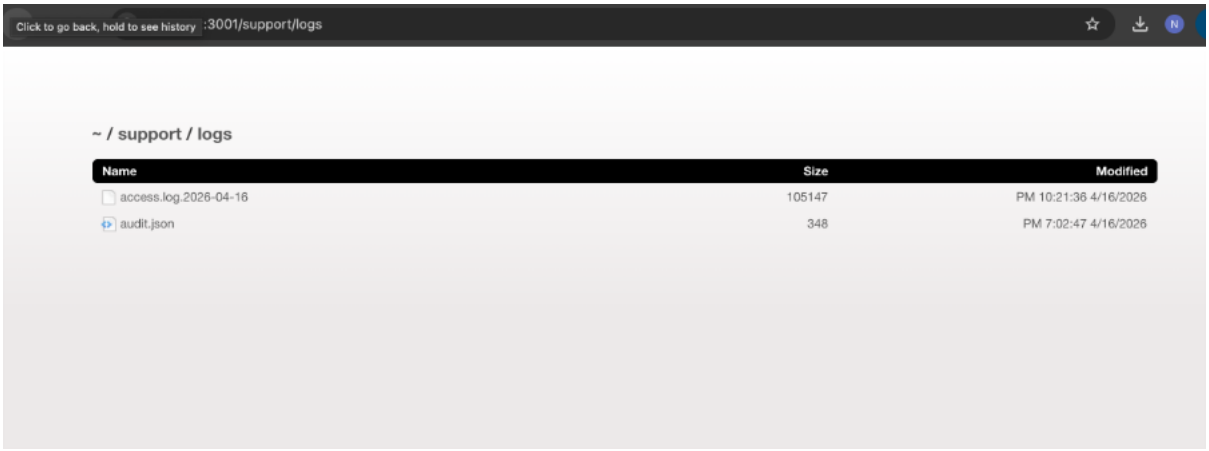
Completed Challenges
OWASP JuiceShop Scoreboard screenshot



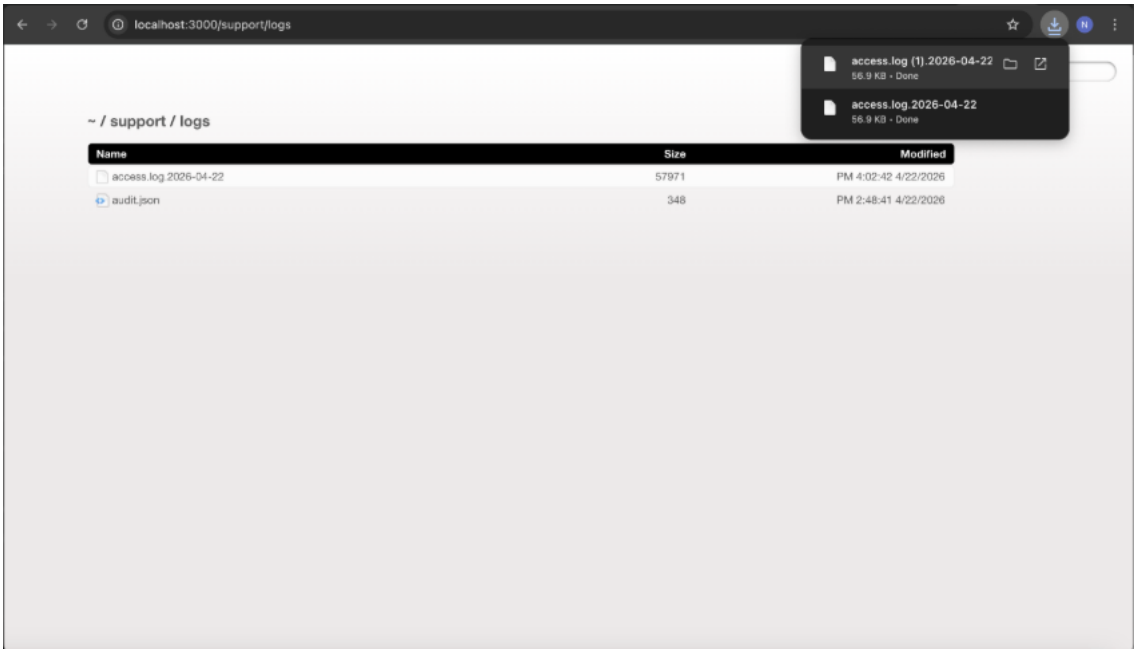
Individual contributions:
Challenge ID: Access Logs

Vulnerability: Security Misconfiguration (Information Disclosure)
Student Responsible: Nana Sarpong

Evidence:



Publicly accessible /support/logs directory showing server log files.



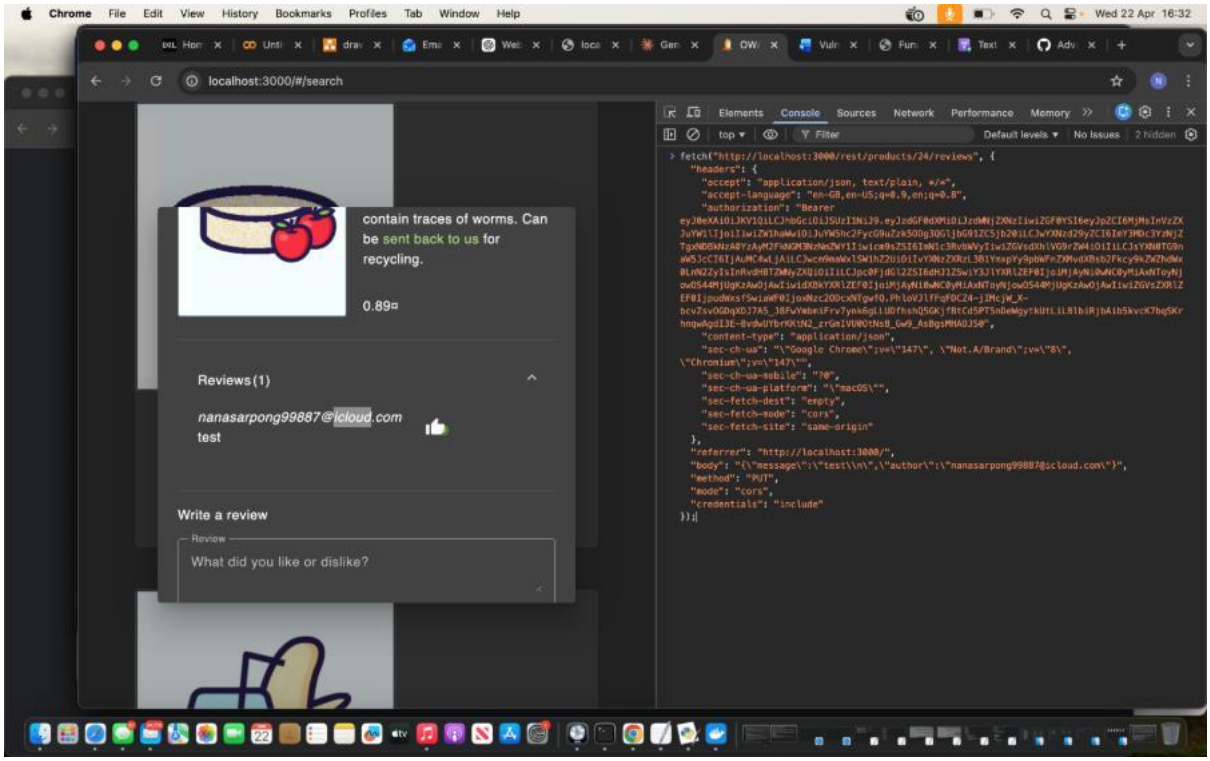
Accessing and viewing log files without authentication.

Mitigation Proposed:

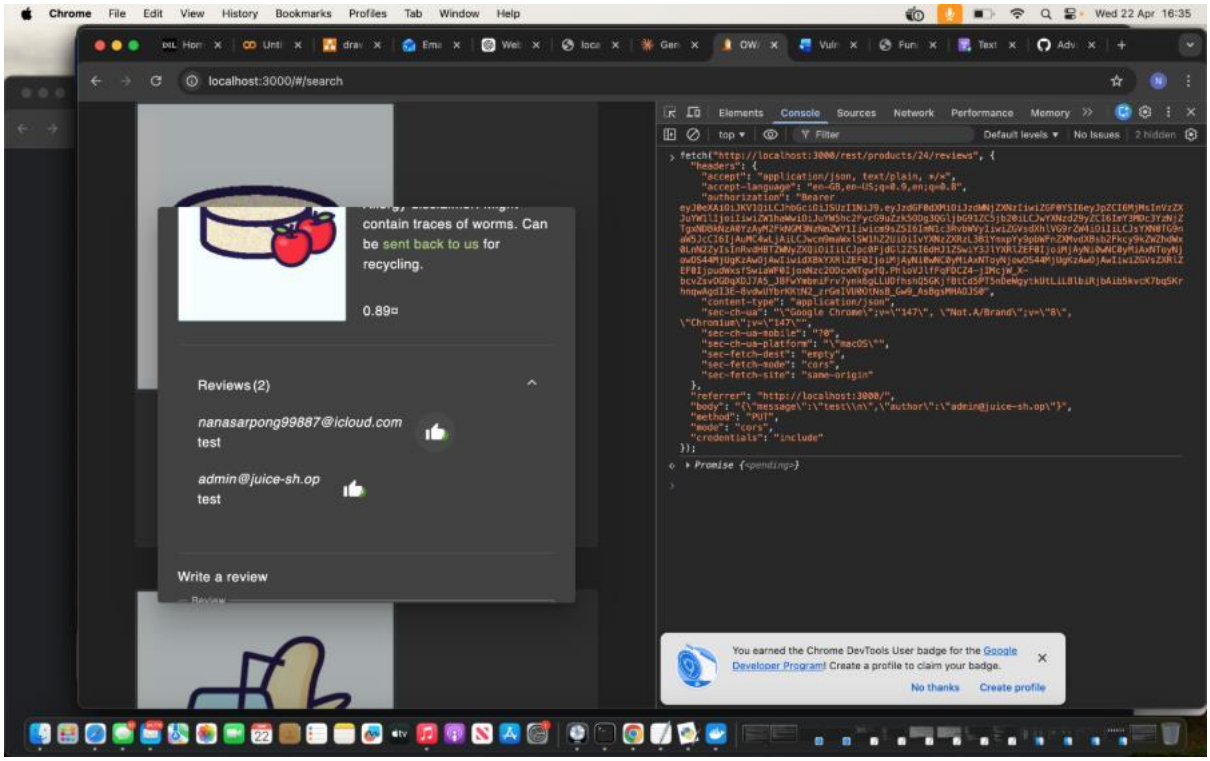
Disable directory indexing
Restrict access to logs using authentication and authorization
Store logs outside the web root
Avoid logging sensitive information
Challenge ID: Forged Review

Vulnerability: Broken Access Control
Student Responsible: Nana Sarpong

Evidence:



Modified request impersonating admin user via author field



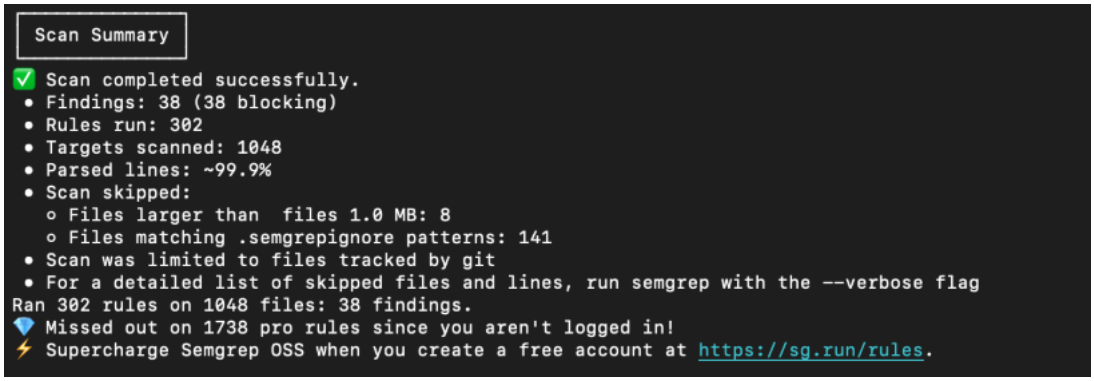
Forged review successfully posted under admin account

Mitigation Proposed:

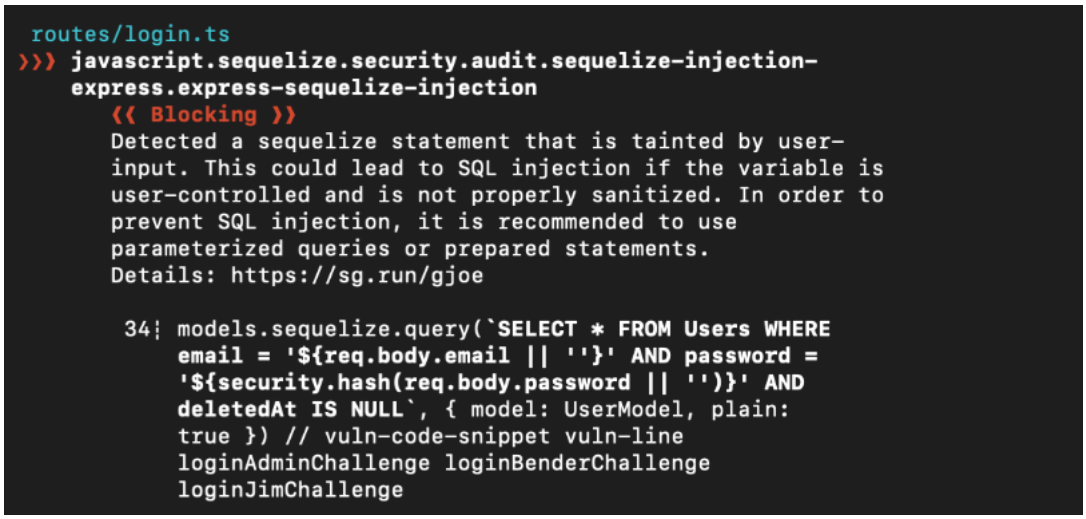
Remove reliance on client-side fields for user identity
Enforce server-side validation of user identity
Use session/token-based authentication
Implement proper authorization checks
SAST Analysis

Tool Used: Semgrep
Student Responsible: Nana Sarpong

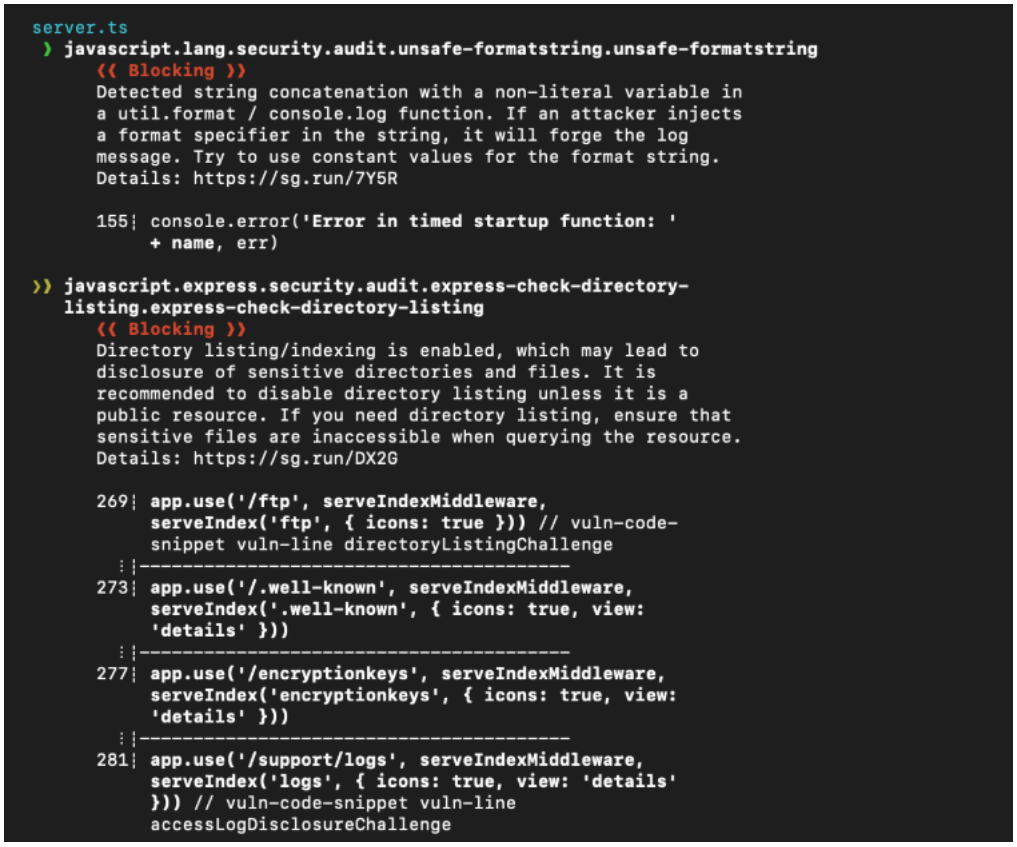
Evidence:



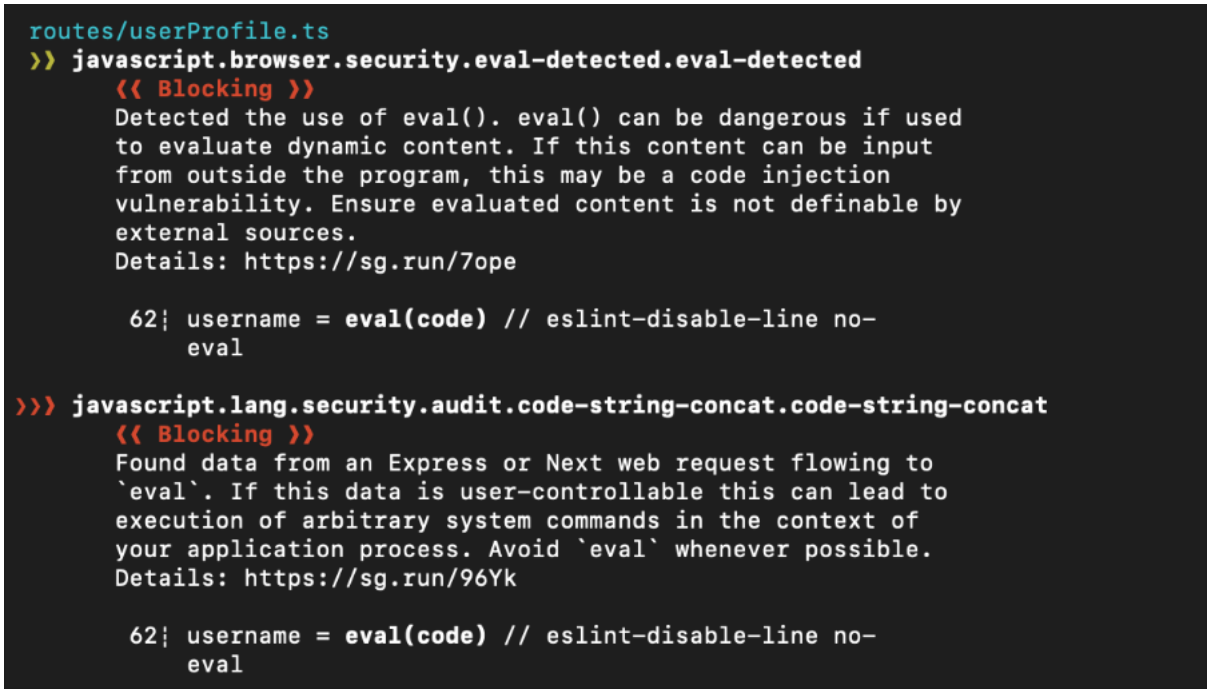
Semgrep scan summary showing 38 detected security findings across the Juice Shop codebase.



Semgrep detection of SQL injection vulnerability in the login route



Directory listing enabled for sensitive directories including application logs



Use of eval() detected in the user profile route, indicating potential code injection risk

Mitigation Proposed:

Use parameterized queries to prevent SQL Injection
Disable directory listing for sensitive folders
Remove use of eval() and replace with safer logic
Follow secure coding practices and OWASP guidelines

Vulnerability Analysis

Nana sarpong

Description and Root Cause

The “Access Log” challenge demonstrates a Security Misconfiguration vulnerability, where server log files are exposed through a publicly accessible endpoint (/support/logs). This aligns with OWASP Top 10 A05: Security Misconfiguration, which includes improper configuration of servers and exposed sensitive resources.

In secure systems, access logs are intended for internal use and should not be accessible to external users. However, in this case, the application allows direct access to log files without authentication.

The root cause is improper configuration of server routes and middleware. Directory indexing is enabled, and a public route is mapped directly to a local logs directory. There are no authentication or authorization controls, resulting in unrestricted access to sensitive data.

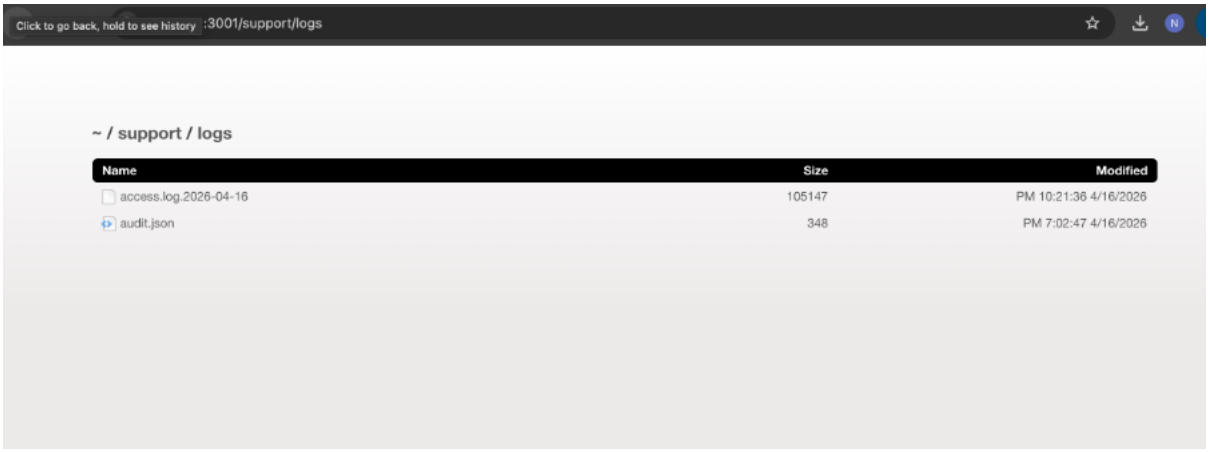
Investigation

The weakness in the system was found by looking closely at the application and checking some common entry points. We took a hint from the challenge and decided to investigate paths like /logs and /support to see if we could find any issues.

When we looked at the /support/logs directory, we found a list of all the log files inside. This showed us that the directory indexing was turned on, which meant that sensitive files were visible to everyone.

Looking deeper into the problem using special tools in the browser, it became clear that requests to this particular endpoint were going through just fine without needing any kind of login or permission. This made it obvious that the real issue here was not some clever hacking trick, but rather a mistake in how things were set up.

Figure: Publicly accessible /support/logs directory showing server log files.

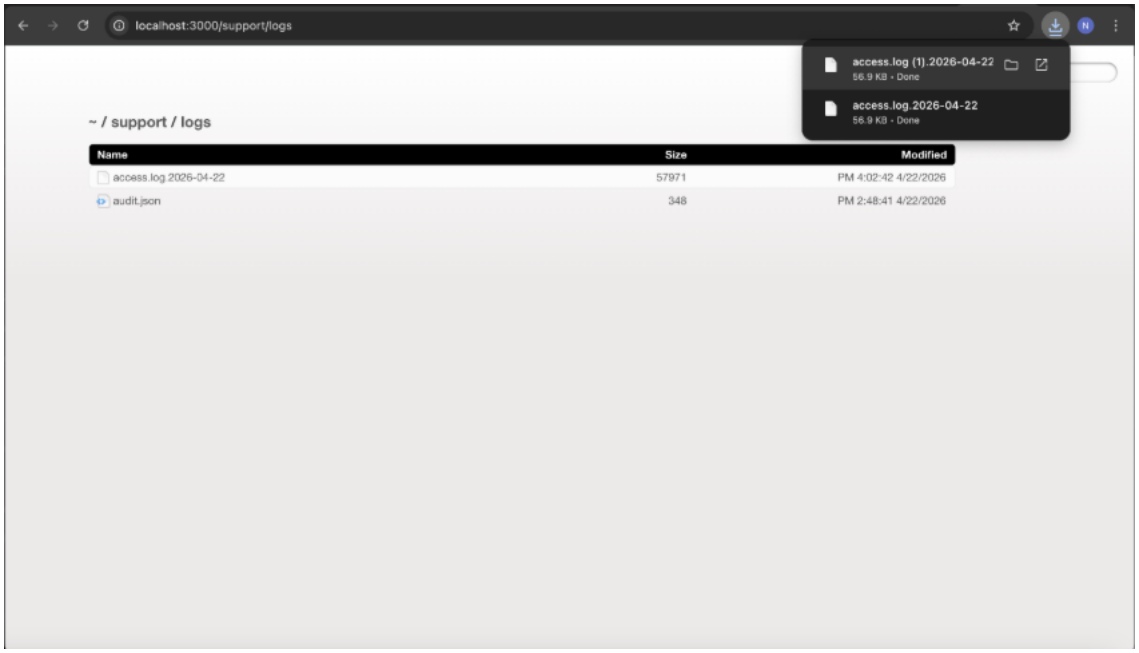


Exploit

The problem is pretty straightforward and doesn't need any fancy tools. All someone has to do to get to the logs is go to:
http://localhost:3000/support/logs

This displays a list of log files, which can be opened or downloaded. The logs may contain:
HTTP request data system information user activity
Since there's nothing to stop them, anyone can get to this information.

Figure: Accessing and viewing log files without authentication.



Impact

Leaving logs out in the open can be really bad for security. One big problem is that it can let attackers see inside the system, learn about its inner workings, and even find out about the endpoints and structure. This can give them a lot of valuable information that they can use to cause harm.

Logs can have private information about users, which can cause problems with privacy. Also, bad people can use this information to find out more about a system, making it easier for them to take advantage of other weaknesses.
This makes the system more vulnerable to attacks and puts sensitive information at risk of being exposed.

Mitigation

To mitigate this vulnerability:
Restrict access to log files (admin-only) Disable directory indexing Store logs outside the web root Implement authentication and authorization Avoid logging sensitive data
Following secure configuration practices and OWASP guidelines can help prevent such issues.

Description and Root Cause

The “Forged Review” challenge in OWASP Juice Shop demonstrates a Broken Access Control vulnerability, where a user can submit a product review while impersonating another user. This aligns with OWASP Top 10 A01: Broken Access Control, which occurs when an application fails to enforce proper restrictions on user actions.

In a secure system, user identity should be determined by the server based on authentication mechanisms such as sessions or tokens. However, in this case, the application relies on a client-controlled parameter (author) included in the request payload. This allows users to modify the value and impersonate other users, including administrators.

The root cause of this vulnerability is the improper trust placed in client-side input. Instead of validating the identity of the user on the server, the application accepts the author field without verification. This results in a failure of authorization controls and violates the principle that users should only be able to perform actions on their own behalf.

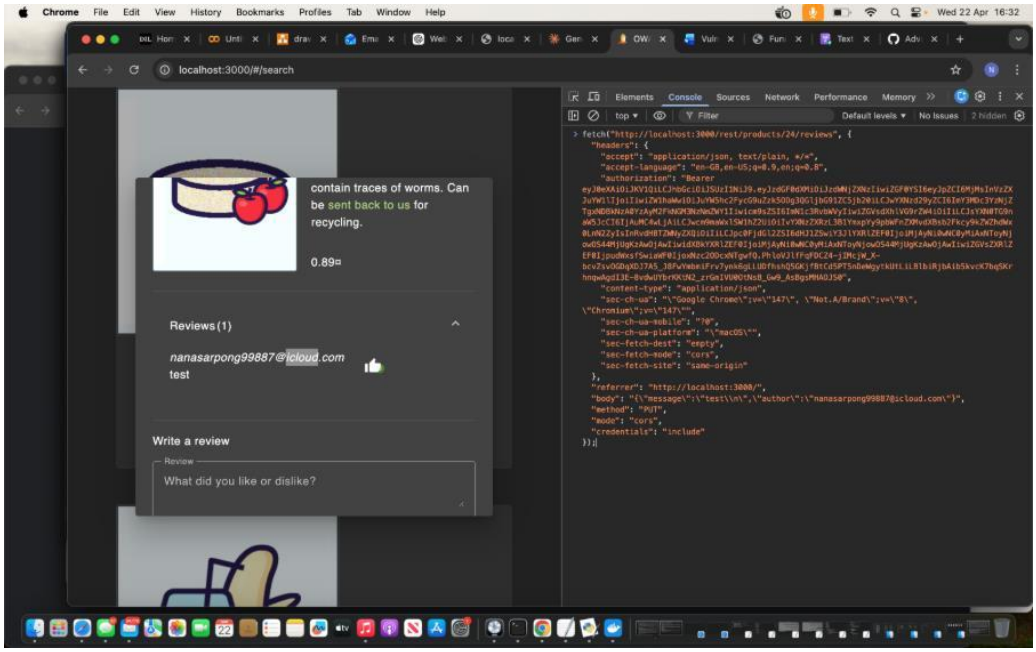
Investigation

The vulnerability was identified by analysing the review submission functionality using browser developer tools. When a review was submitted, a request was captured in the Network tab, revealing a JSON payload containing fields such as message and author.

The presence of the author field raised suspicion, as it contained the email of the logged-in user. This suggested that the application might not be validating the value on the server side. To confirm this, the request was copied and modified using the browser console. The author value was changed to another user’s email address, such as admin@juice-sh.op, before resending the request. The application accepted the modified request, indicating that no server-side validation was performed.

This approach reflects DAST (Dynamic Application Security Testing), where vulnerabilities are identified by observing application behaviour during execution.

Figure: Modified request impersonating admin user via author field.

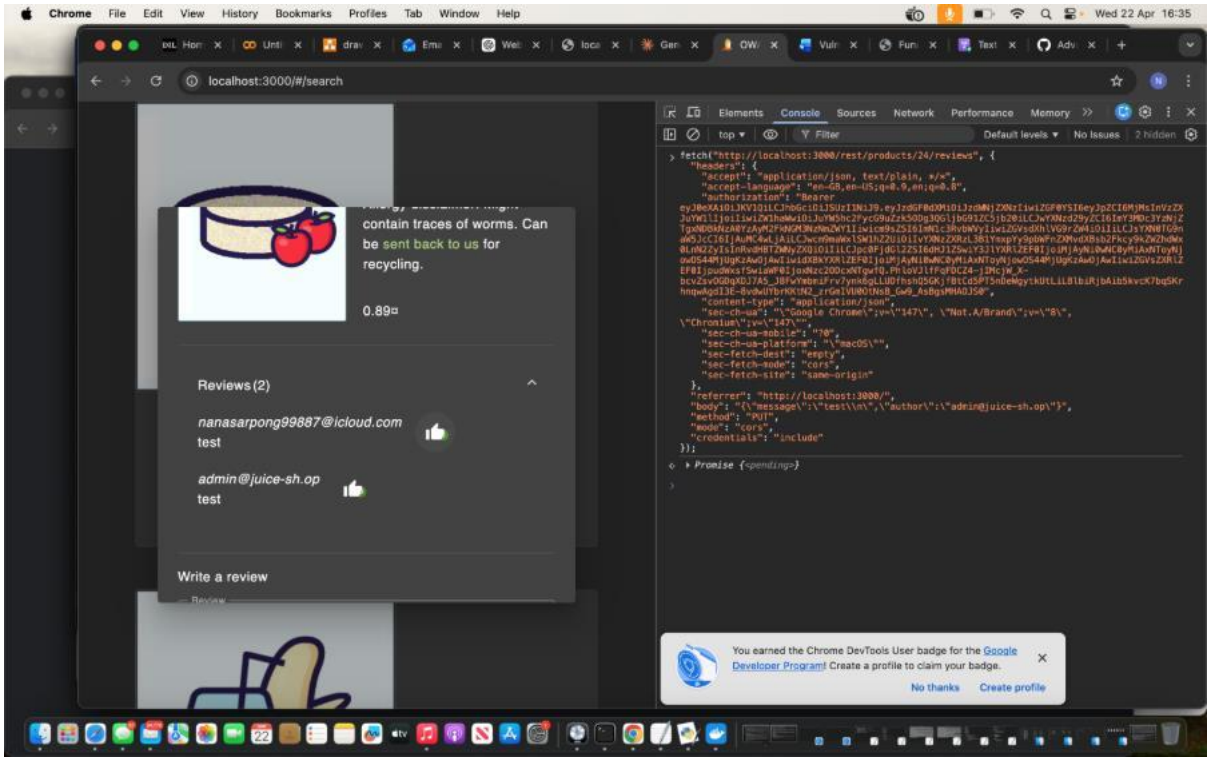


Exploit

The exploit involves intercepting a legitimate request and modifying its payload. After submitting a normal review, the request is captured and edited to change the author field: { "message": "forged review", "author": "admin@juice-sh.op" }

The changed request is sent to the server again. Because the application doesn't check if the author is the same as the authenticated user, the request is handled without any issues. As a result, the review is shown under the name of the user who was impersonated, which clearly shows a problem with access control. This means that someone could pretend to be another user and post reviews that look like they came from that user.

Figure: Forged review successfully posted under admin account.



Impact

This vulnerability allows attackers to impersonate other users, including administrators. This can lead to:

posting misleading or malicious content damaging user trust and platform credibility
potential abuse of privileges

It represents a horizontal access control failure, where users can perform actions on behalf of others at the same privilege level.

In real-world systems, similar flaws could allow attackers to modify sensitive data or perform unauthorized actions.

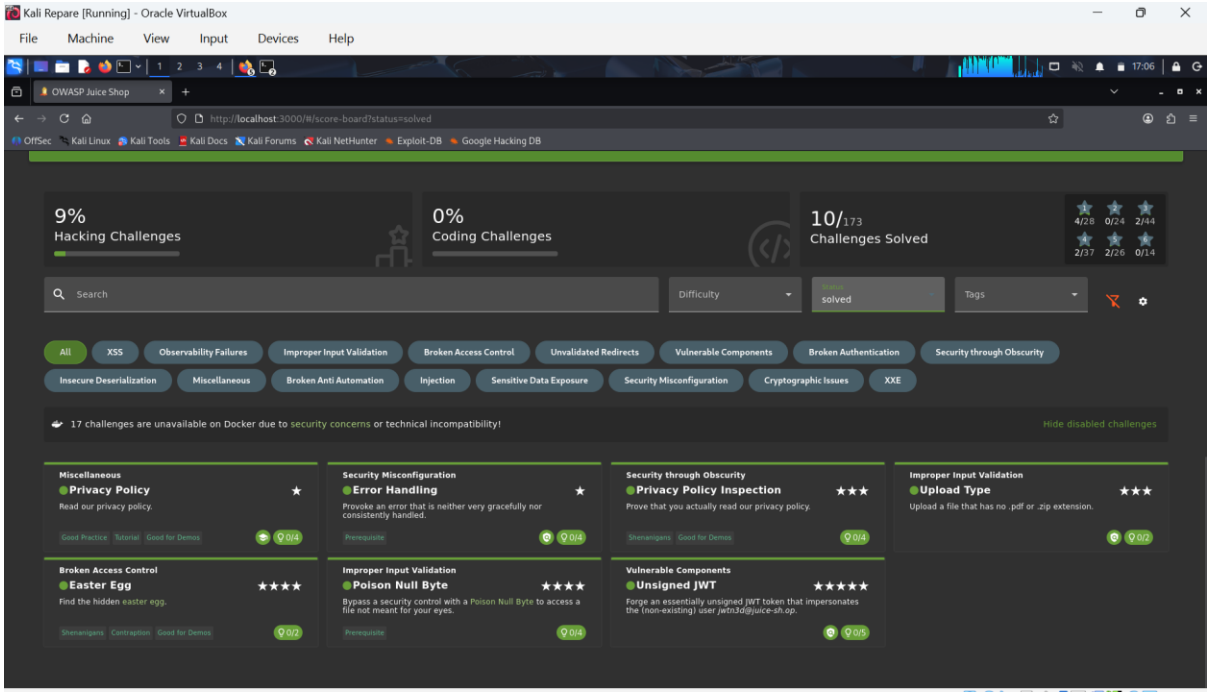
Mitigation

To mitigate this vulnerability:
User identity must be derived from the server-side session, not client input The author field should be removed or ignored Implement strict authorization checks Validate all incoming data Perform regular security testing (SAST and DAST)

These steps make sure that people can't pretend to be someone else and that only the right people can get in. This way, everything stays safe and secure

---
IBRAHIMA SOW

Completed Challenges
OWASP JuiceShop Scoreboard screenshot



Poison Null Byte:

a.**Vulnerability description and root cause**:

The Easter Egg challenge in OWASP Juice Shop demonstrates an Improper Input Validation vulnerability through a classic Poison Null Byte attack. The application hosts a hidden file named eastere.gg within the /ftp directory, but direct access is restricted because the server only permits files with .md or .pdf extensions. When the file was requested normally, the server returned a 403 Error: Only .md and .pdf files are allowed!, indicating that validation was based primarily on the visible file extension.

The root cause lies in insufficient server-side sanitization of user-supplied file paths. By appending a URL-encoded null byte (%2500, decoded as %00) followed by .md, the application interpreted the input inconsistently. The validation mechanism accepted the .md extension, while the underlying file handling logic truncated the string at the null byte, effectively accessing eastere.gg instead of eastere.gg.md. This mismatch between validation and file system interpretation allowed a restricted file to be accessed. The flaw reflects insecure handling of special characters and legacy null-termination behavior.

b.**Rational or investigative process**:

The investigation began by manually exploring the /ftp directory after discovering accessible file listings. This enumeration revealed several files, including the suspicious eastere.gg, which appeared intentionally unusual. Attempting direct access to /ftp/eastere.gg triggered an extension restriction error, confirming that the application enforced a file-type whitelist.
This behavior suggested that the security control focused on file extension validation rather than securely resolving file paths. Based on common web exploitation techniques and Juice Shop challenge patterns, a Poison Null Byte bypass was tested by appending %2500.md to the URL. This payload was designed to satisfy the extension check while potentially truncating the actual filename during processing. The successful retrieval of the hidden file confirmed the presence of null byte injection and validated the hypothesis.

c.**How the exploit works**:

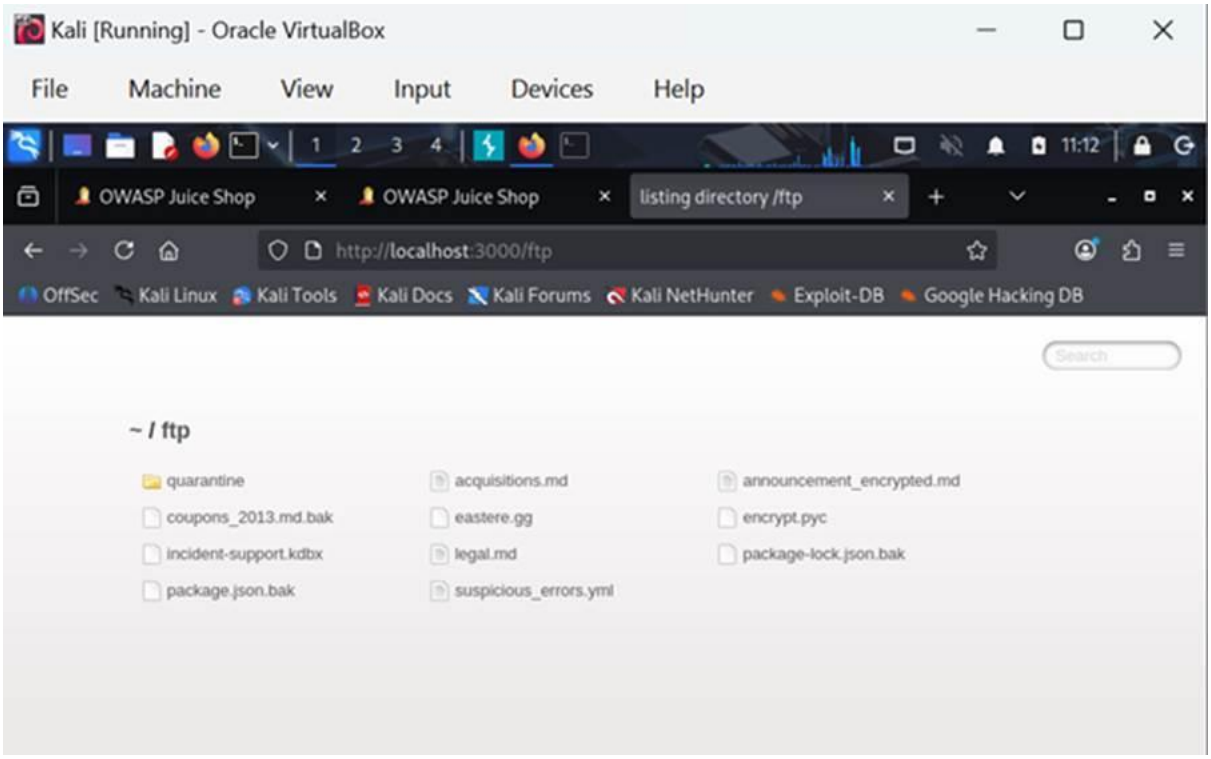
The exploit works by abusing null byte injection to manipulate how the application processes file names. The payload used was:
http://localhost:3000/ftp/eastere.gg%2500.md
Here, %25 represents the encoded %, resulting in %00 after decoding. The server-side validation interprets the request as ending in .md, which satisfies the whitelist requirement. However, when the path is processed internally, the null byte acts as a string terminator, causing the application to ignore everything after eastere.gg. As a result, the protected file is served despite extension restrictions.

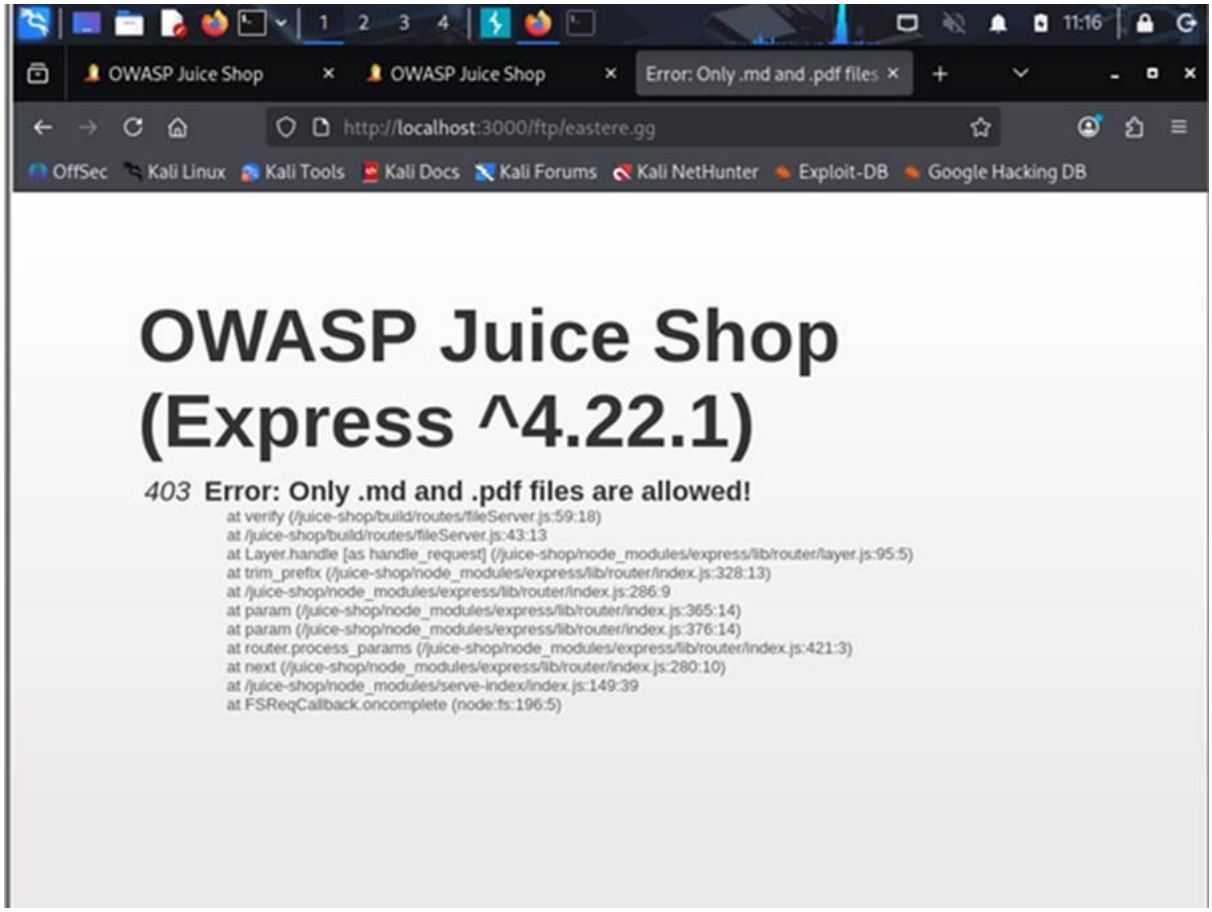
d.Potential impact:

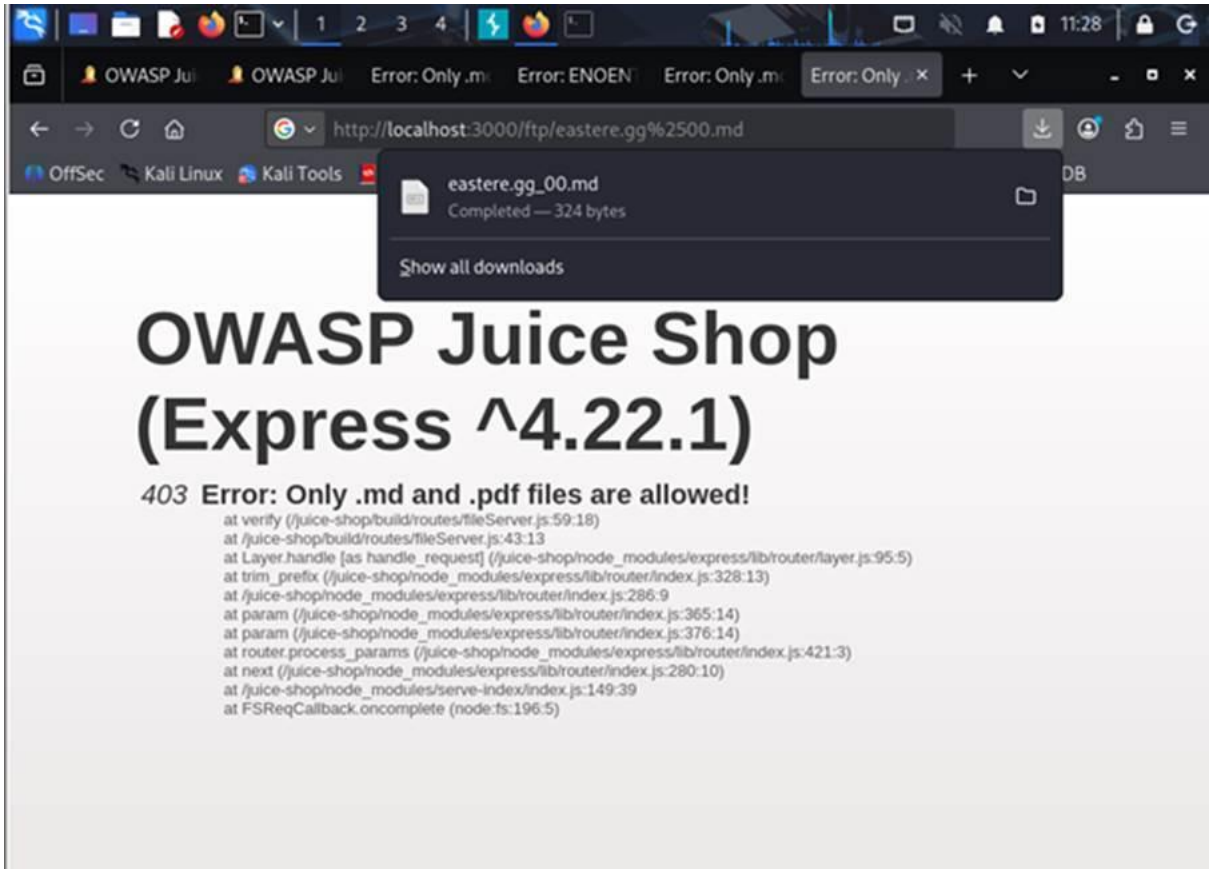
In real-world applications, this vulnerability could allow attackers to bypass file access restrictions and retrieve sensitive resources not intended for public exposure. Examples include backup files, configuration files, source code, or internal documents. Exposure of such files could lead to information disclosure, credential leakage, or further compromise through reconnaissance. Although this challenge demonstrates the flaw in a controlled environment, similar weaknesses in production systems could significantly expand the attack surface.

e.Recommended mitigation:

To mitigate this vulnerability, applications should implement strict server-side canonicalization and validation of file paths before access decisions are made. User input should be sanitized to reject null bytes and other special characters entirely. File extension checks should occur only after proper normalization of the requested path. Additionally, access controls should rely on explicit allowlists of approved files or directories rather than simple extension matching alone. Using secure frameworks that correctly handle encoded characters and disabling legacy null byte interpretation are also essential to prevent this class of attack.




OWASP Juice Shop
Error: Only .md and .pdf files
http://localhost:3000/ftp/eastere.gg
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
OWASP Juice Shop (Express ^4.22.1)
403 Error: Only .md and .pdf files are allowed!
at verify (/juice-shop/build/routes/fileServer.js:59:18)
at /juice-shop/build/routes/fileServer.js:43:13
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at param (/juice-shop/node_modules/express/lib/router/index.js:365:14)
at param (/juice-shop/node_modules/express/lib/router/index.js:376:14)
at router.process_params (/juice-shop/node_modules/express/lib/router/index.js:421:3)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/node_modules/serve-index/index.js:149:39
at FSReqCallback.oncomplete (node:fs:196:5)


http://localhost:3000/ftp/eastere.gg%2500.md
eastere.gg_00.md
Completed — 324 bytes
Show all downloads
OWASP Juice Shop (Express ^4.22.1)
403 Error: Only .md and .pdf files are allowed!
at verify (/juice-shop/build/routes/fileServer.js:59:18)
at /juice-shop/build/routes/fileServer.js:43:13
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at param (/juice-shop/node_modules/express/lib/router/index.js:365:14)
at param (/juice-shop/node_modules/express/lib/router/index.js:376:14)
at router.process_params (/juice-shop/node_modules/express/lib/router/index.js:421:3)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/node_modules/serve-index/index.js:149:39
at FSReqCallback.oncomplete (node:fs:196:5)

Upload Type:

a. Vulnerability description and root cause:
The “Upload Type” challenge in OWASP Juice Shop demonstrates an Improper Input Validation vulnerability within the complaint file upload functionality. The application is intended to restrict uploads to .pdf or .zip files only, suggesting that users should only submit approved document formats as invoice attachments. However, this restriction is not securely enforced on the server side. Instead, the validation largely depends on client-side controls such as the browser interface, file extension checks, or MIME type declarations sent by the user’s browser. Because these controls can be manipulated, an attacker can intercept the HTTP request and modify the uploaded filename or Content-Type header to bypass restrictions.

The root cause is insufficient server-side validation of uploaded files. Rather than verifying the actual file content (magic bytes), securely checking file signatures, or enforcing a strict allowlist after upload, the application appears to trust user-supplied metadata such as filename extensions and MIME headers. Since these fields are fully controllable by the client, they should never be considered authoritative for security decisions. This design flaw creates an opportunity for malicious users to upload unauthorized file types such as executable files, scripts, or disguised malware. In real-world applications, such weaknesses can lead to unrestricted file upload vulnerabilities, which are often severe because attackers may use them to distribute malicious content, bypass business rules, or potentially achieve remote code execution depending on server configuration.

b. Rational or investigative process:
The investigative process began by examining the Complaint form in Juice Shop, which explicitly stated that only .pdf or .zip files were permitted. This immediately suggested input validation controls related to file type enforcement. Based on the challenge hints, the upload mechanism itself required deeper inspection to determine whether validation occurred solely on the client side or was properly enforced by the server.
A legitimate PDF file (legit.pdf) was first selected to ensure the upload request would be accepted by the browser interface. Burp Suite Proxy and Repeater were then used to intercept the HTTP POST request to the /file-upload endpoint. Within the multipart form-data request, the filename parameter and Content-Type header were inspected. These fields were manually altered from:
filename="legit.pdf"
Content-Type: application/pdf
to:

filename="malware.exe"
Content-Type: application/x-msdownload
This modification tested whether the server validated actual content or merely trusted client input. After sending the manipulated request through Burp Repeater, the server returned HTTP/1.1 204 No Content, indicating successful processing despite the unauthorized file type. This confirmed that validation controls were inadequate and primarily dependent on modifiable client-side parameters.

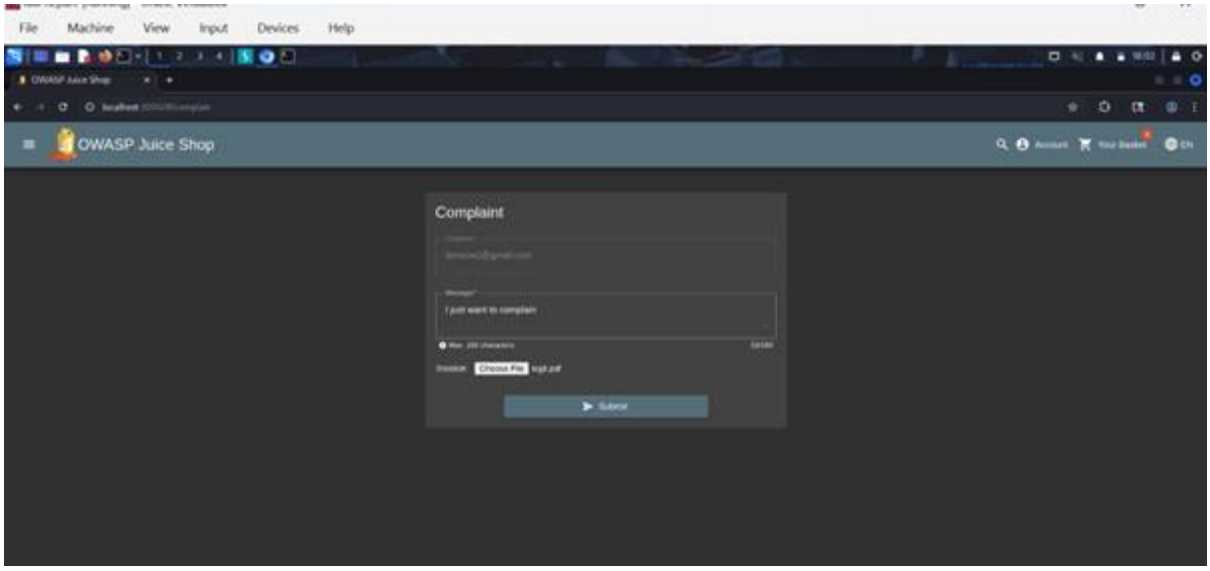
c. How the exploit works:
The exploit works by abusing the trust placed in user-controlled HTTP request components. During a normal upload, the browser submits multipart form-data containing the chosen file name and MIME type. By intercepting this request before it reaches the server, an attacker can alter these values without changing the visible browser-side restrictions.

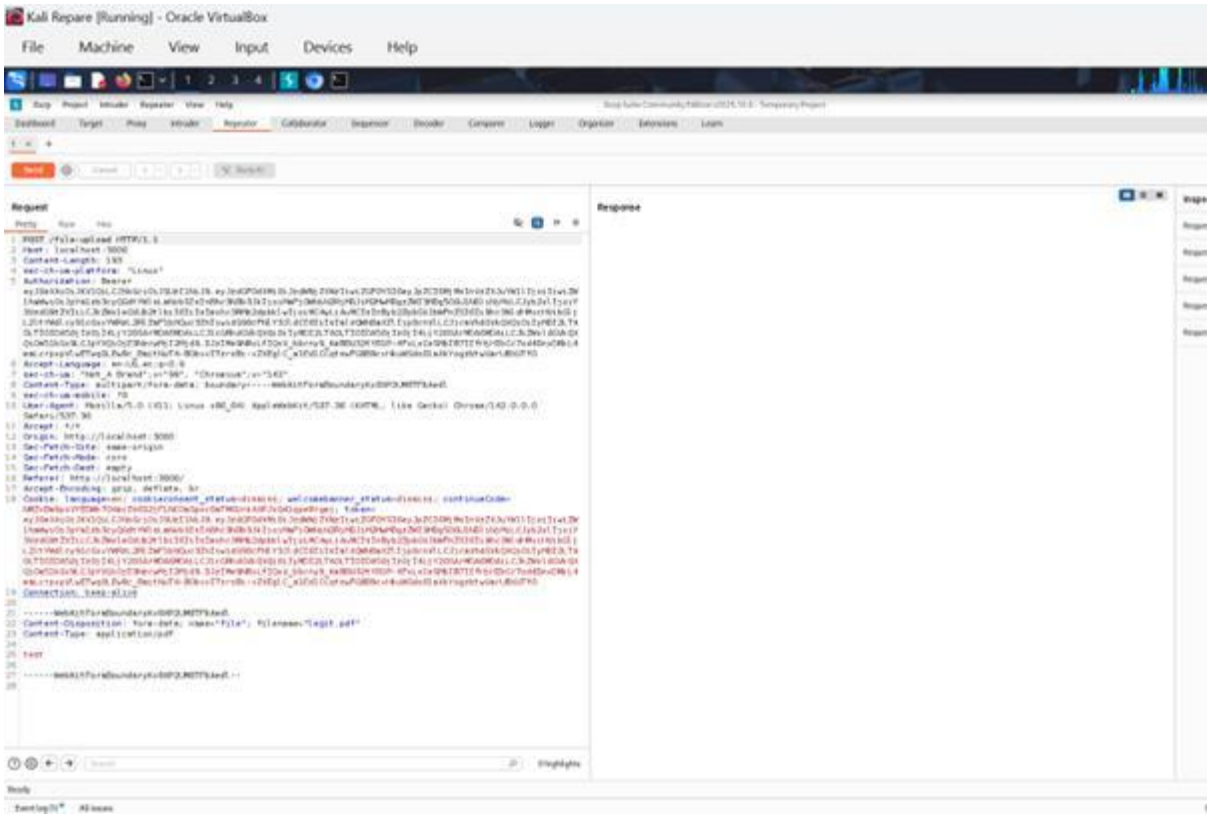
Because the server accepted the modified request despite the .exe extension, it demonstrated that backend validation did not properly inspect the true nature of the uploaded file. The attack bypasses extension restrictions and MIME filtering by directly tampering with the request. This allows unauthorized files to be uploaded even when the user interface appears restrictive.

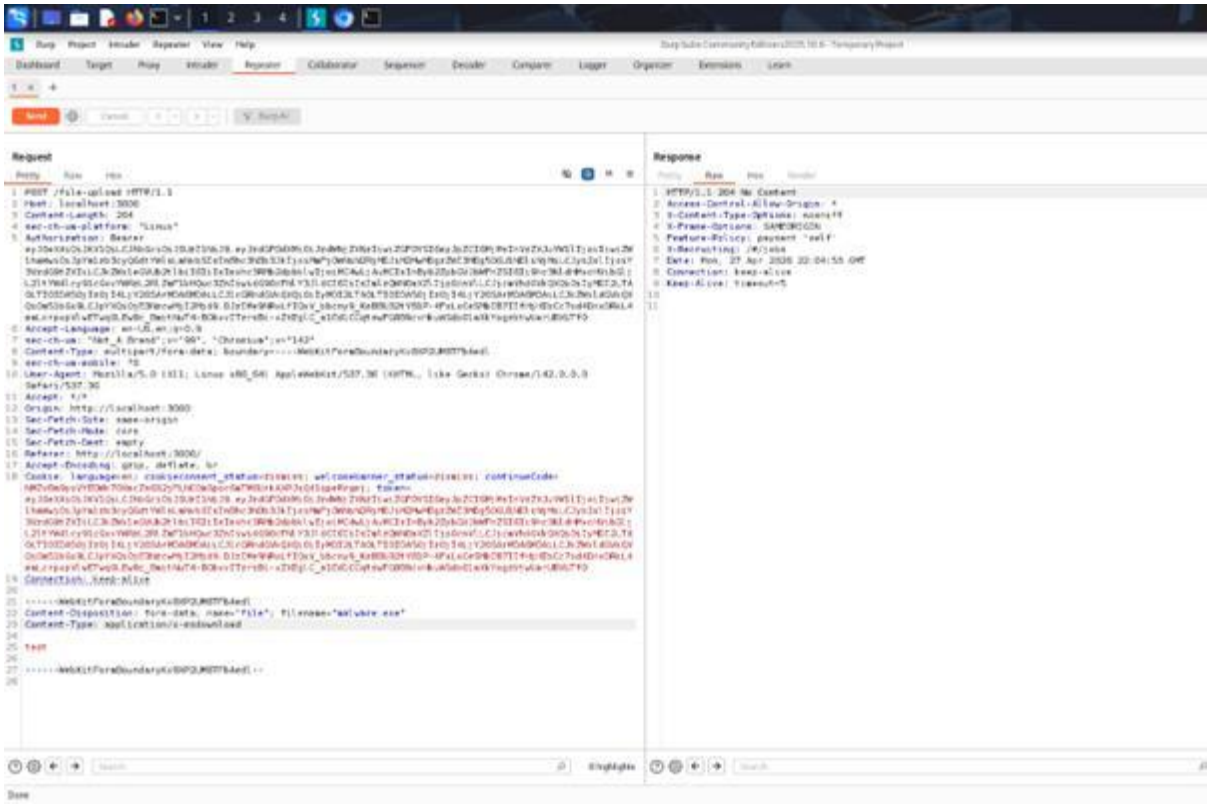
d. Potential impact:
In a real-world environment, improper file upload validation can have serious consequences. Attackers could upload malicious executables, web shells, or scripts that may later be executed on the server or downloaded by other users. This could lead to malware distribution, phishing, stored XSS, server compromise, or remote code execution depending on storage and execution settings. Even if direct execution is not possible, unrestricted uploads may still violate business policies, consume storage resources, or expose internal systems to harmful payloads. Because file upload features often interact with backend storage systems, weak validation significantly expands the attack surface.

e. Recommended mitigation:
To mitigate this vulnerability, validation must be enforced server-side rather than relying on client-side restrictions. The server should implement strict allowlisting of approved file types and verify actual file signatures (magic bytes) instead of trusting extensions or MIME headers. Uploaded files should be renamed server-side, stored outside

executable directories, scanned for malware, and subject to size restrictions. Content Security Policy and secure storage permissions should also prevent uploaded files from being executed directly. Logging and monitoring suspicious uploads can further strengthen detection. By combining these measures, organizations can significantly reduce the risk of unrestricted or malicious file upload attacks.







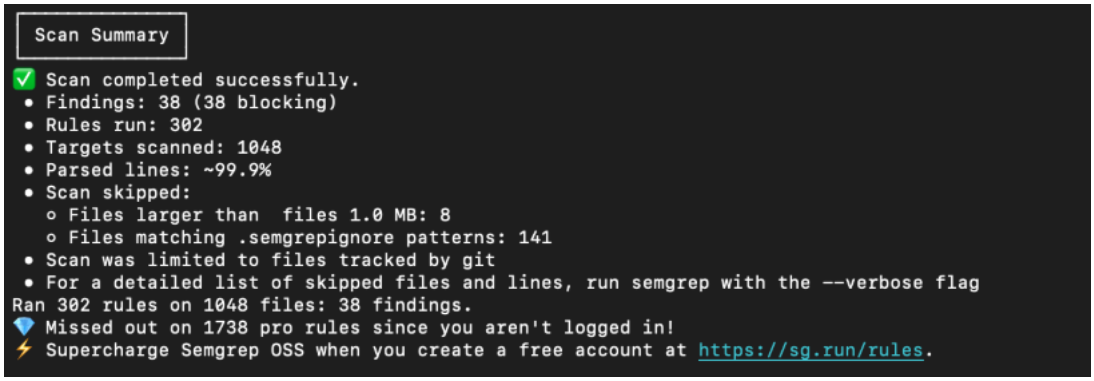
SAST Testing

Methodology

A Static Application Security Testing (SAST) analysis was conducted on the OWASP Juice Shop source code using the Semgrep security analysis tool. Semgrep was selected due to its ability to detect common vulnerabilities in JavaScript and Node.js applications through pattern-based static analysis. The scan was executed locally on the full Juice Shop codebase using the default rule configuration (--config=auto), which applies a comprehensive set of security rules targeting vulnerabilities such as injection flaws, insecure coding practices, and misconfigurations.

This analysis aimed to find security flaws in the application's source code without actually running the app. By doing so, it's possible to catch vulnerabilities early on by looking at how user input moves through the system and affects sensitive areas like database queries, file access, and code execution. This approach helps identify potential weaknesses before they can be exploited, making it a crucial step in securing the application.

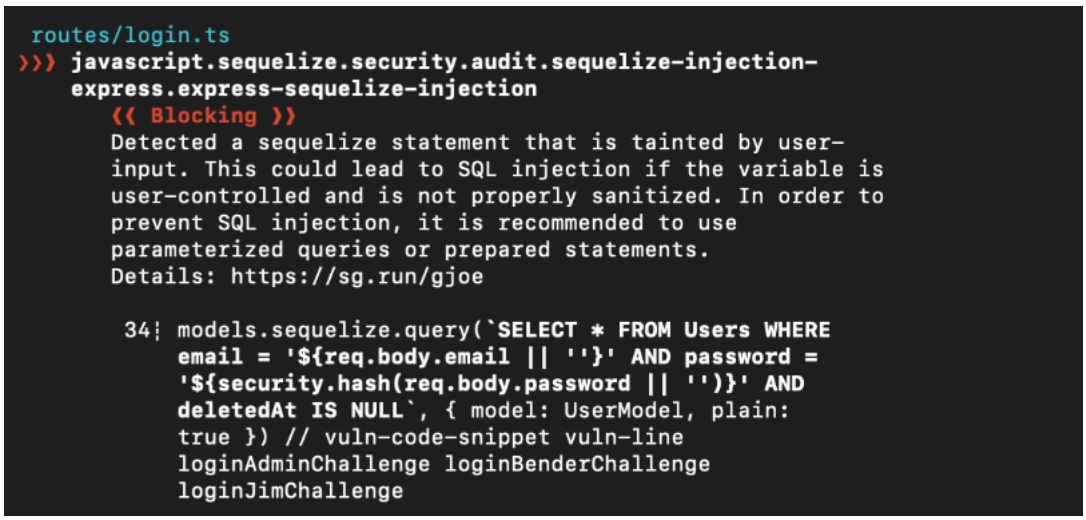
Figure: Semgrep scan summary showing 38 detected security findings across the Juice Shop codebase.



The scan was completed without any problems and it found 38 issues that could block things from working properly. These issues were looked at closely and filtered to find the most important and serious vulnerabilities that needed to be analyzed further. Three main issues were picked because of how severe they were, how easily they could be exploited in the real world, and how they fit into the well-known OWASP Top 10 categories.

Key Findings

Figure: Semgrep detection of SQL injection vulnerability in the login route



Description and Root Cause

The first critical vulnerability identified is a SQL Injection flaw within the login functionality of the application. The issue occurs in the routes/login.ts file, where a raw SQL query is constructed using direct string concatenation of user-supplied input.

The relevant code snippet is shown below:

```
models.sequelize.query(`SELECT * FROM Users WHERE
email = '${req.body.email || ''}' AND password =
'${security.hash(req.body.password || '')}' AND
deletedAt IS NULL`, { model: UserModel, plain: true })
```

This code directly embeds user input (req.body.email and req.body.password) into the SQL query string without proper sanitisation or parameterisation. This creates a situation where an attacker can manipulate the input fields to alter the structure of the SQL query.

The root cause of this vulnerability is the use of unsafe string concatenation when building database queries, combined with the absence of input validation or parameterized query mechanisms.

Risk and Impact

SQL Injection is a high-severity vulnerability and is listed in the OWASP Top 10 under Injection flaws. An attacker could exploit this vulnerability to:

Bypass authentication mechanisms Access unauthorized user accounts Extract sensitive data from the database Modify or delete database records

For instance, someone with bad intentions could enter special SQL commands to change the way the query works, which would let them get into the system without using the right login information.

Validation of Finding

This problem is a real concern because the information that the user enters is directly used in the SQL query without being checked or protected. The tool that checks for security issues in the code, called SAST, correctly finds the unsafe way that data is passed from the user's input to the database.

Mitigation

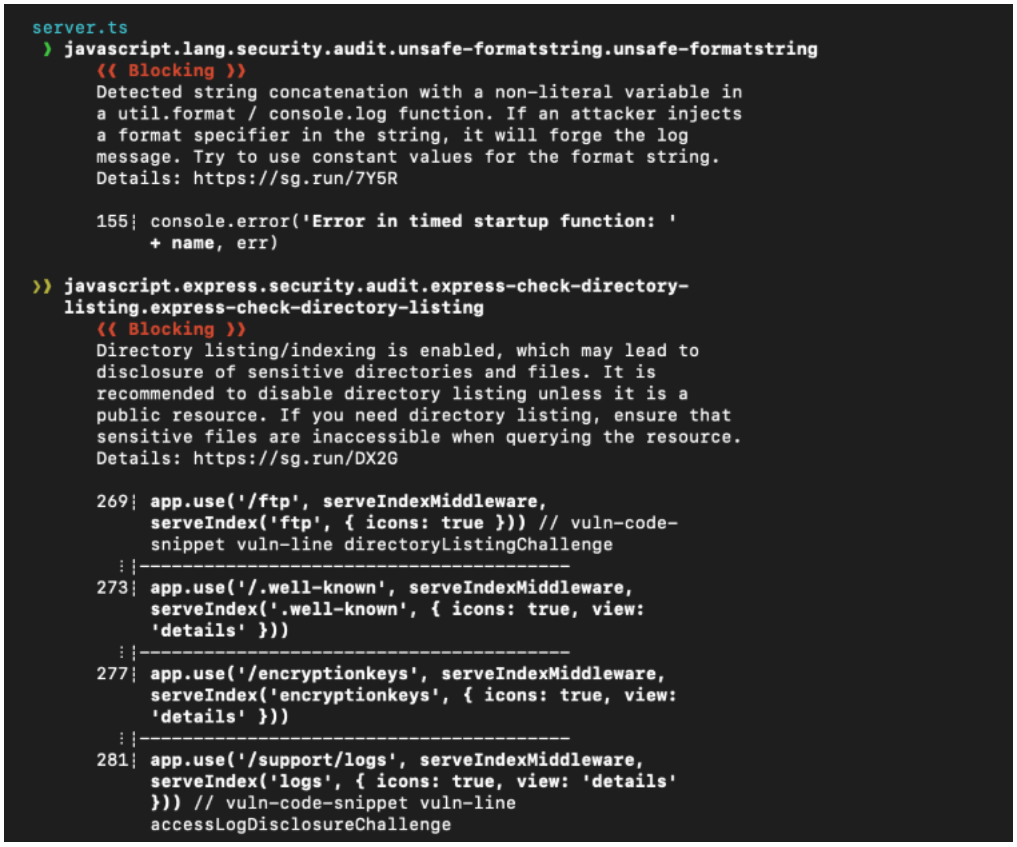
To prevent SQL Injection, the application should:

Use parameterized queries or prepared statements Avoid constructing SQL queries using string concatenation Use ORM-safe methods provided by Sequelize Implement strict input validation and sanitisation

These steps make sure that what the user types in is only used as information, not as instructions that the computer has to follow.

The root cause of this vulnerability is the use of unsafe string concatenation when building database queries, combined with the absence of input validation or parameterized query mechanisms.

Figure: Directory listing enabled for sensitive directories including application logs.



Description and Root Cause

Another problem they found was with the security settings, which weren't done right. This made some sensitive directories visible, which is a big deal. The issue is in the server.ts file, where they turned on directory indexing for a few directories, like /support/logs. This means that people can see what's in those directories, which isn't good. They should probably turn off directory indexing to keep those directories private.

The relevant code is:

app.use('/support/logs', serveIndexMiddleware,
serveIndex('logs', { icons: true, view: 'details' }))

This setup lets users look through the logs directory right from the web application. It also has special routes that help serve files from these directories in a dynamic way.

The root cause of this vulnerability is the intentional enabling of directory listing without restricting access to sensitive resources.

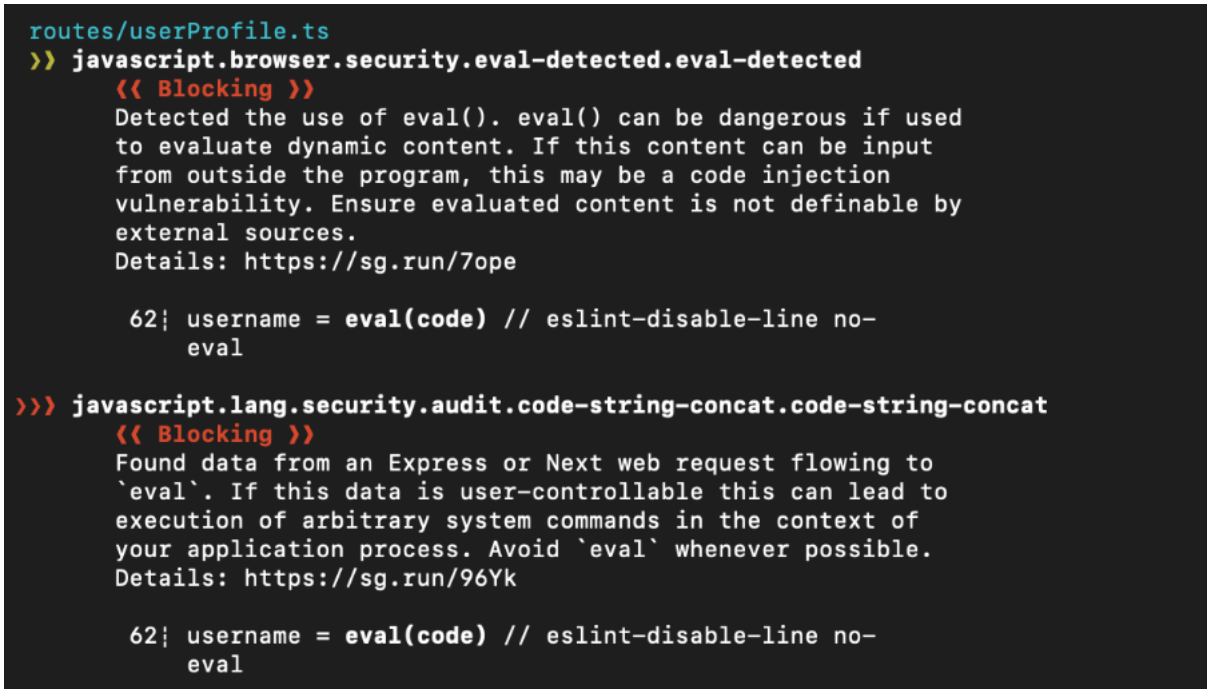
Risk and Impact

This is a big security problem, it's called Security Misconfiguration and Information Disclosure, which is one of the top issues listed by OWASP. When log files are made public, it can have serious consequences because these files often have sensitive information like: User activity information IP addresses System errors Potentially sensitive debugging information
If someone with bad intentions finds out about this weakness, they can use it to learn more about how the application works on the inside. This can help them plan even more attacks, like trying to figure out more secrets or getting special access to things they shouldn't have. This weakness is closely related to the "Access Log" challenge in Juice Shop, where users can actually get their hands on log files right through the app's interface.

Validation of Finding

This is a confirmed true positive. The directory listing is explicitly enabled in the code, and no access controls are enforced. This means any user can access the logs if they know or discover the endpoint.

Mitigation
To mitigate this issue, the application should:
Disable directory listing for all sensitive directories Restrict access using authentication and authorization mechanisms Store log files outside publicly accessible directories Implement proper access control policies
These steps make sure that important system information is kept safe from people who shouldn't see it.
Figure: Use of eval() detected in the user profile route, indicating potential code injection risk



Description and Root Cause

The third vulnerability involves the use of the eval() function in the routes/userProfile.ts file. The following line was flagged by Semgrep:

username = eval(code)

The eval() function executes JavaScript code represented as a string. While this can be useful in certain scenarios, it is extremely dangerous when used with untrusted or user-controlled input.

The root cause of this vulnerability is the use of dynamic code execution without validating or restricting the input being evaluated.

Risk and Impact

This vulnerability falls under Code Injection and Unsafe Function Usage. The use of eval() introduces the risk of arbitrary code execution, which is one of the most severe types of vulnerabilities.

If an attacker is able to control the input passed to eval(), they could:

Execute malicious JavaScript code
Access or modify sensitive data
Take control of application logic
Potentially escalate privileges

This makes the weakness extremely serious and hazardous in actual situations, posing a significant threat to security and stability.

Validation of Finding

This result is a true positive because the eval() function is used directly without any protection. The SAST tool is right to identify this as a high-risk pattern, especially when it comes to web applications where user input can be controlled by people outside the system.

Mitigation

To eliminate this vulnerability, the application should:

Avoid using eval() entirely
Replace dynamic execution with safer alternatives
Use strict input validation and whitelisting if dynamic behavior is required
Apply secure coding practices that prevent execution of arbitrary code

Removing eval() significantly reduces the risk of code injection attacks.

Summary of Findings

The SAST analysis revealed several critical vulnerabilities within the OWASP Juice Shop application, including SQL Injection, insecure configuration leading to information disclosure, and unsafe dynamic code execution. These issues highlight common security weaknesses such as improper input handling, lack of access controls, and unsafe programming practices.

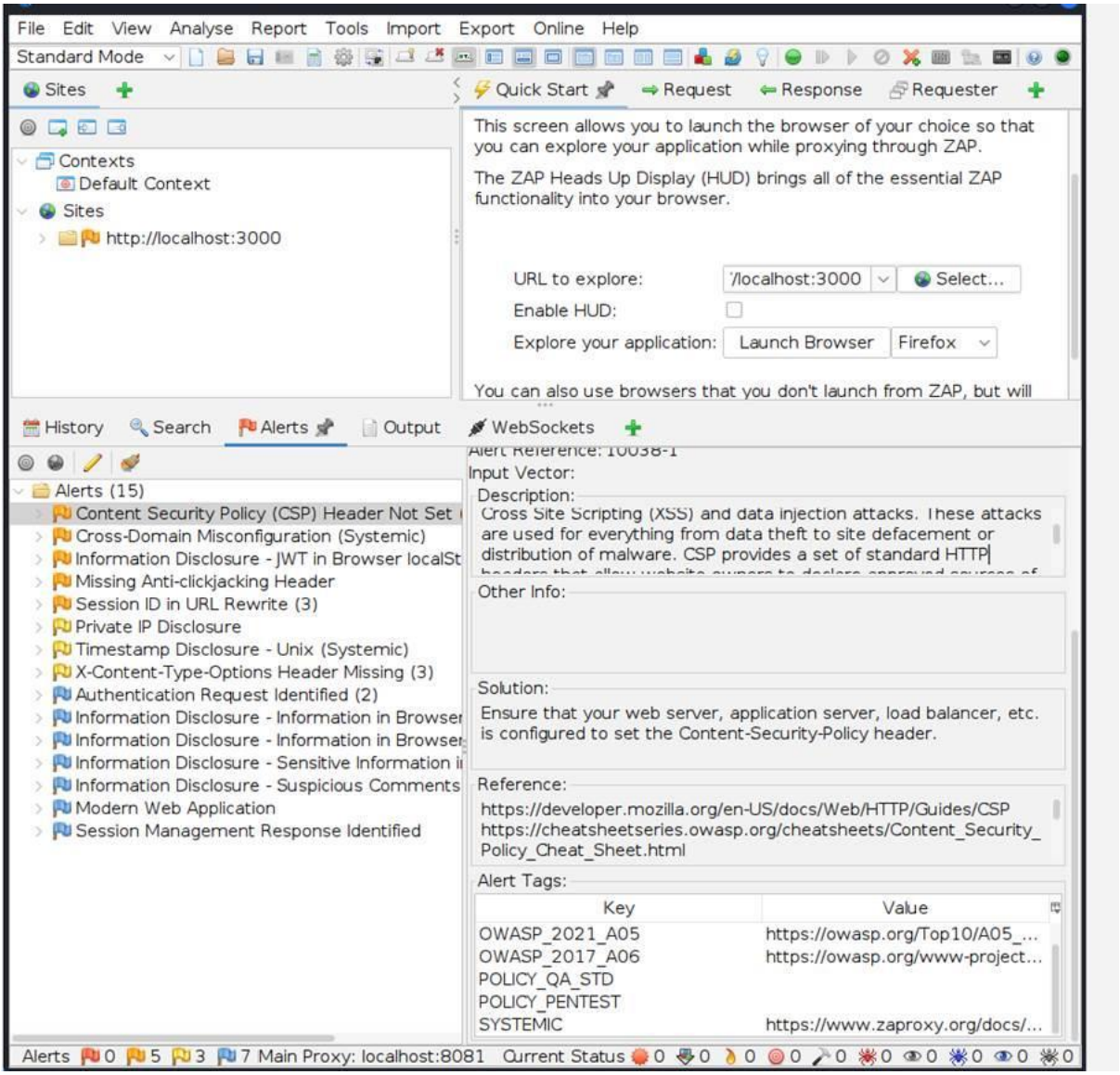
The results show that using static analysis tools, such as Semgrep, can really help find security weaknesses early on, when the code is still being written. To fix these problems, it's crucial to follow secure coding practices, check user input carefully, and manage configurations properly. This approach is key to reducing the risk of attacks on the application. By doing so, developers can make their software more secure and less vulnerable to threats.

DAST Testing

Methodology and analyse of Dynamic Application Security Testing (DAST)

The Dynamic Application Security Testing (DAST) was realised on my AWASP Juice Shop local instance, run on http://localhost:3000 using OWASP ZAP (Zed Attack Proxy) as the main analysis tool. Following the project instructions, evaluation was made by performing a controlled and targeted scan to avoid excessive or long-running automated scans. Instead of performing an analysis of the whole application, the scope of the test has been limited to the most security-relevant components, especially the home page, authentication (login and password), the search item function, as well as some client-side mechanisms such as local storage and specific endpoints such as socket.io. That approach allowed to reduce the noise generated by low-relevance alerts, better control scan duration, and focus the analysis on attack surfaces linked to important OWASP Top 10 categories, particularly Security Misconfiguration, Identification and Authentication Failures, and Session Management Weaknesses.

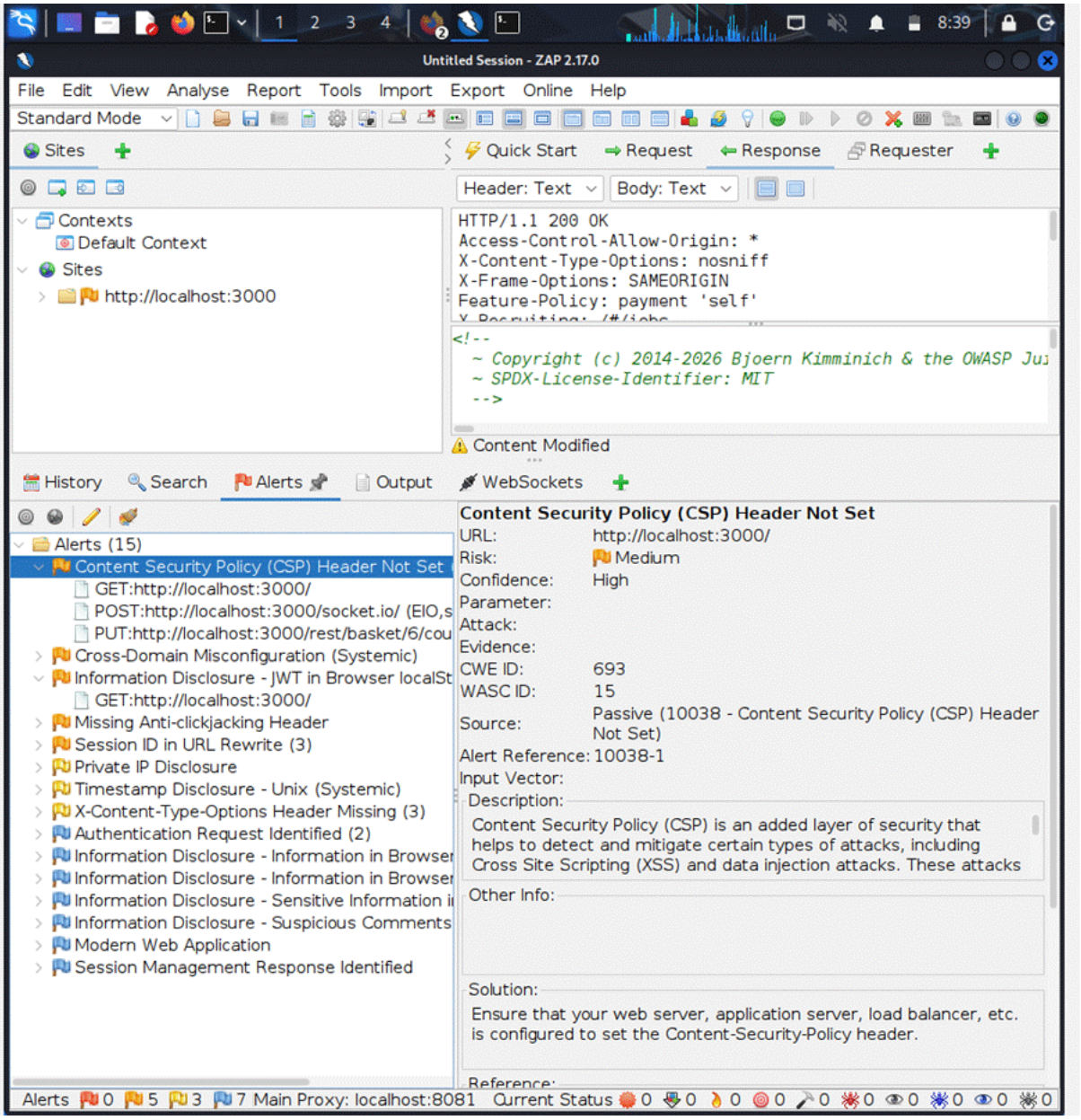
OWASP ZAP was configured mainly in standard mode with a combination of Manual Explore and passive scanning. Instead of using a Full Scan potentially too broad, navigating through out the application was done manually in order to generate a real traffic while allowing for ZAP to intercept and analyse requests et HTTP responds. That method allowed to observe the real behaviour of the application in in targeted usage scenarios. ZAP's Request/Response panels were used to manually inspect HTTP headers, while Firefox Developer Tools were used to verify certain browser-side behaviours, particularly JWT storage in localStorage. That hybrid approach was essential to interpret the results of the scanner, filter the false-positive, and manually to validate the most significant findings.

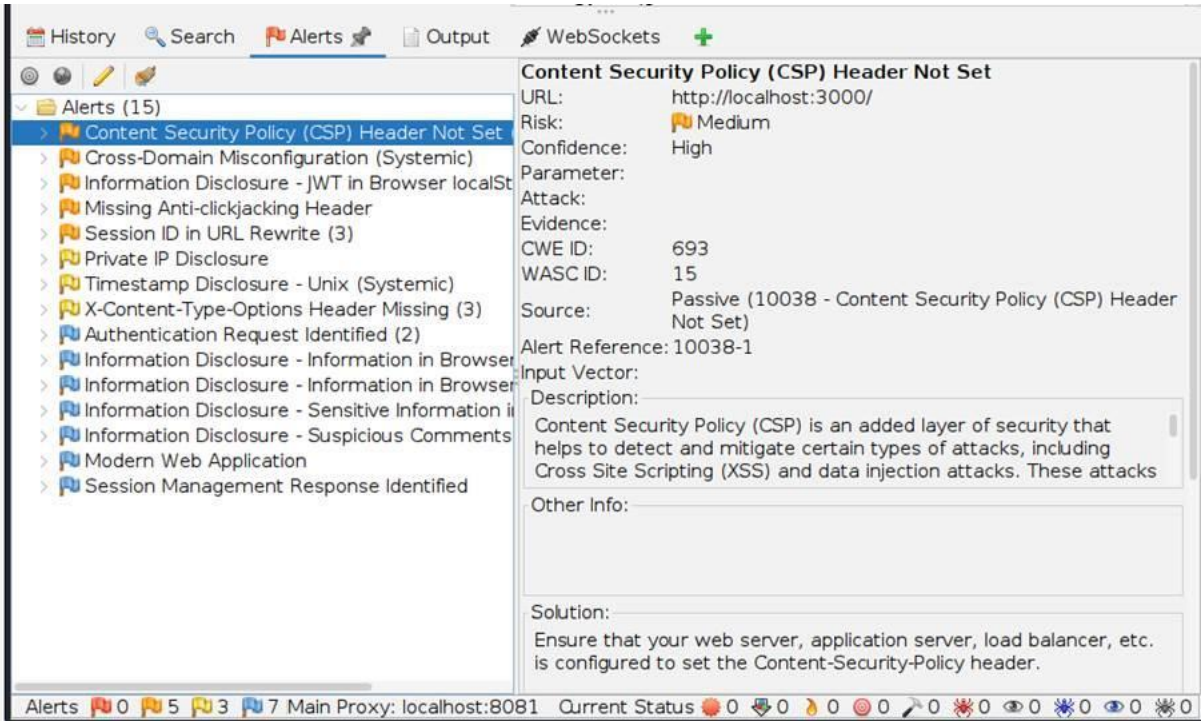


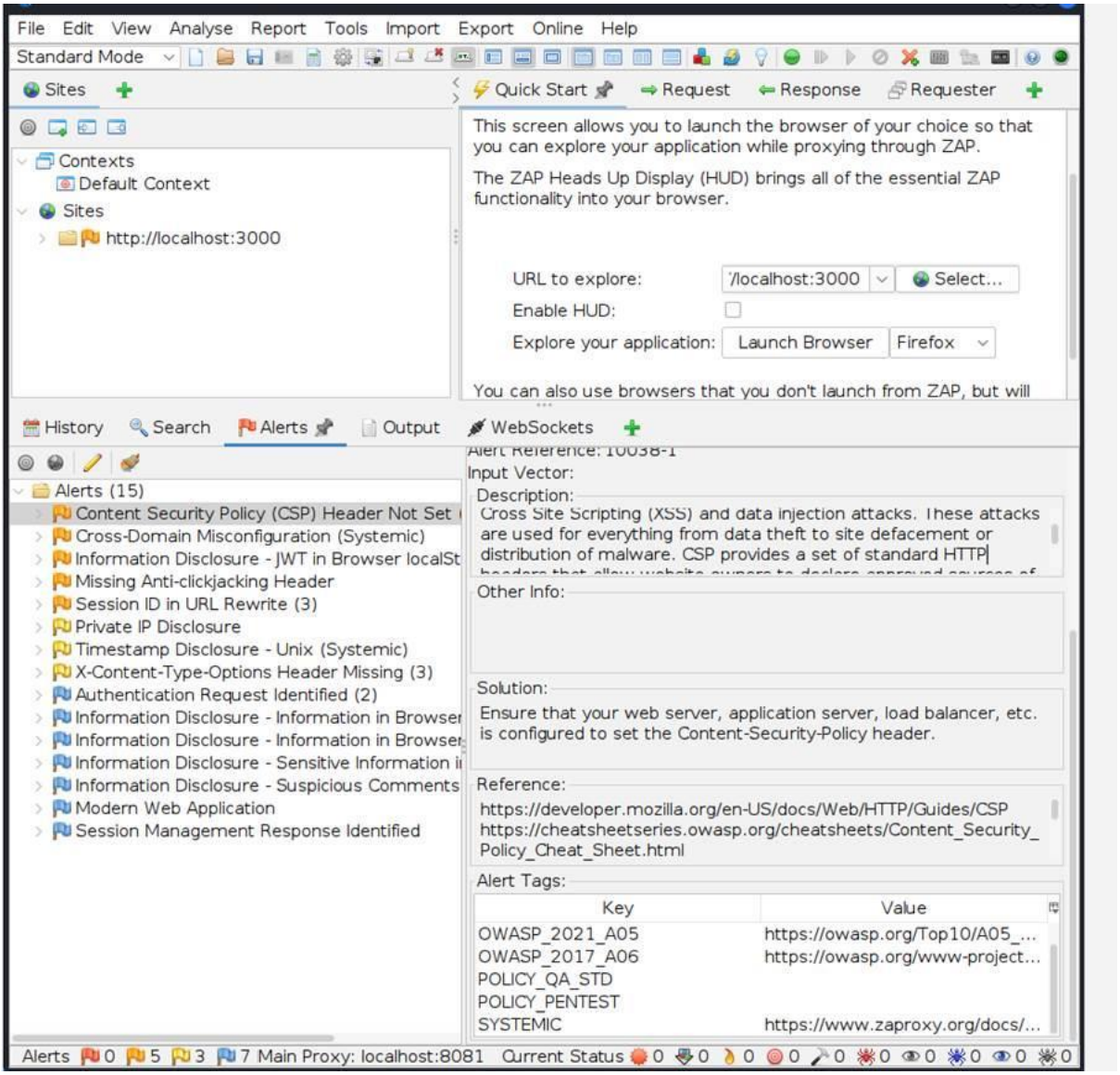
Finding 1: (CSP Header Not Set)

One of the most pertinent results of the scan was Missing of Header of Content Security Policy (**CSP Header Not Set**) on the main page of the application. ZAP identified that weakness as a medium-risk vulnerability associated with CWE-693 (Protection

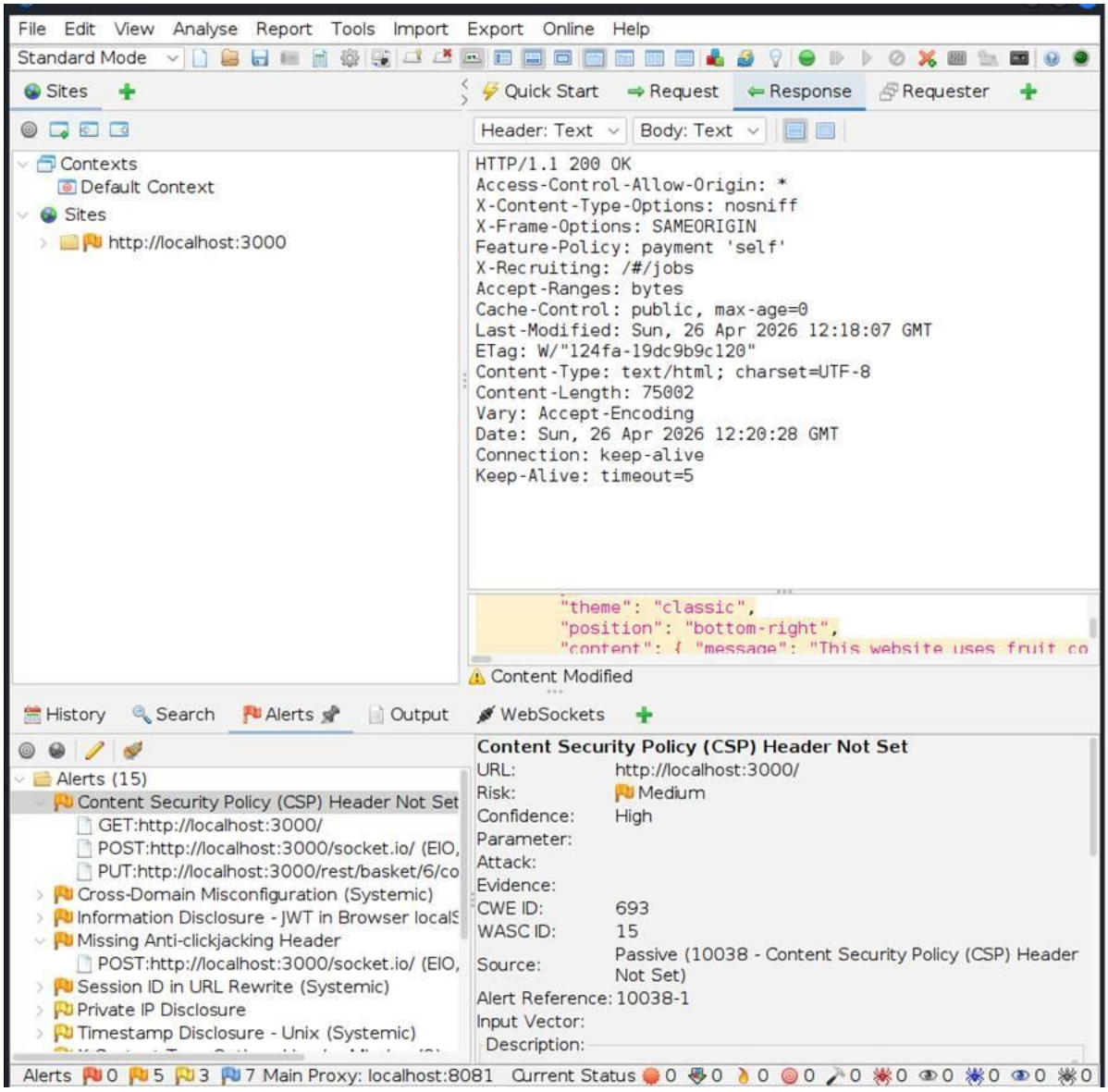
Mechanism Failure). A manual validation was realised by directly inspecting HTTP responds headers in the ZAP Response Header tab. This verification showed that certain security headers, such as X-Frame-Options: SAMEORIGIN and X-Content-Type-Options: nosniff, were present, but that no Content-Security-Policy header was defined. Missing of CSP does not create directly a XXS vulnerability, but it considerably reduces browser protection mechanisms, in case of injection of malicious scripts. Without this security layer, the potential impact of an XSS vulnerability becomes greater, as the browser imposes fewer restrictions on the execution of unauthorized scripts. In an application like Juice Shop, which is heavily client-side oriented, this weakness therefore increases the overall exposure to injection-based attacks. The recommended mitigation is to implement a strict CSP policy, for example default-src 'self'; script-src 'self';, in order to restrict script execution to explicitly authorized sources.




History
Search
Alerts
Output
WebSockets
Alerts (15)
Content Security Policy (CSP) Header Not Set
Cross-Domain Misconfiguration (Systemic)
Information Disclosure - JWT in Browser localSt
Missing Anti-clickjacking Header
Session ID in URL Rewrite (3)
Private IP Disclosure
Timestamp Disclosure - Unix (Systemic)
X-Content-Type-Options Header Missing (3)
Authentication Request Identified (2)
Information Disclosure - Information in Browser
Information Disclosure - Information in Browser
Information Disclosure - Sensitive Information i
Information Disclosure - Suspicious Comments
Modern Web Application
Session Management Response Identified
Content Security Policy (CSP) Header Not Set
URL: http://localhost:3000/
Risk: Medium
Confidence: High
Parameter:
Attack:
Evidence:
CWE ID: 693
WASC ID: 15
Source: Passive (10038 - Content Security Policy (CSP) Header Not Set)
Alert Reference: 10038-1
Input Vector:
Description:
Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks
Other Info:
Solution:
Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.
Alerts 0 5 3 7 Main Proxy: localhost:8081 Current Status 0 0 0 0 0 0 0 0 0 0



Manual Validation:

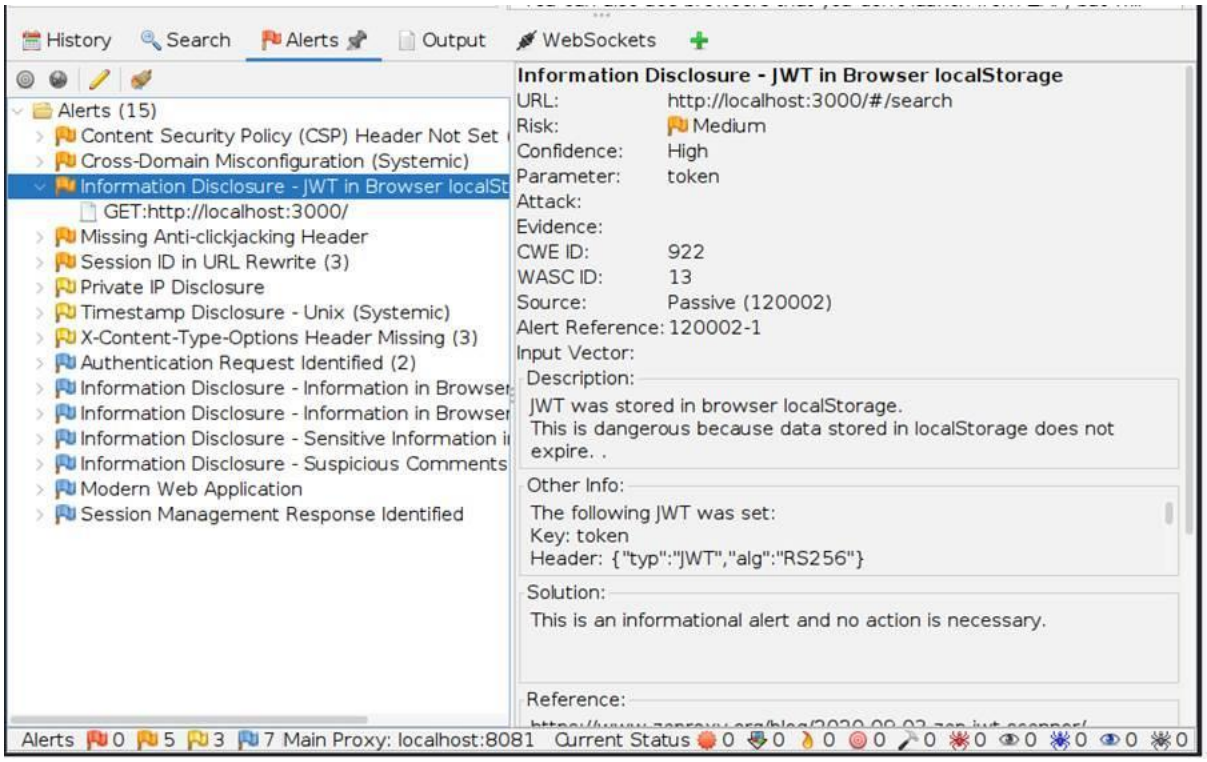


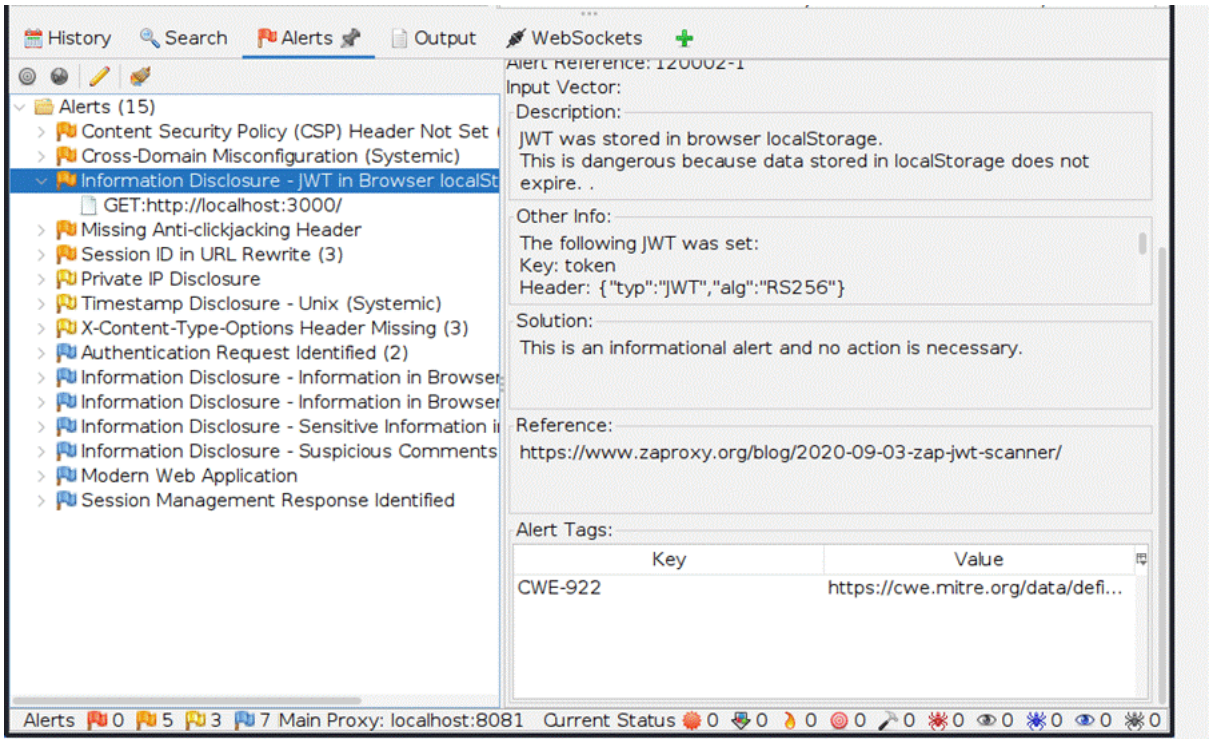
Finding 2: Information Disclosure-JWT in browser localstorage

A second finding particularly significant concerned the storage of the JWT (JSON Web Token) in the browser's localStorage. ZAP detected the presence of a token in localStorage under key token, which was validated manually via Firefox Delveloper Tools using localStorage.getItem("token") command. That verification confirmed that the token authentication was directly accessible via JavaScript client-side.

That behaviour does not, by itself, constitute an immediate compromise, but it represents a significant risk in the event of an XSS vulnerability or the execution of malicious scripts in the browser. Since localStorage is accessible via JavaScript and the data it contains does not benefit from HttpOnly protections, an attacker capable of executing code in the browser could potentially steal the token and hijack the user session. That risk becomes even more relevant when it is combined with missing of CSP,

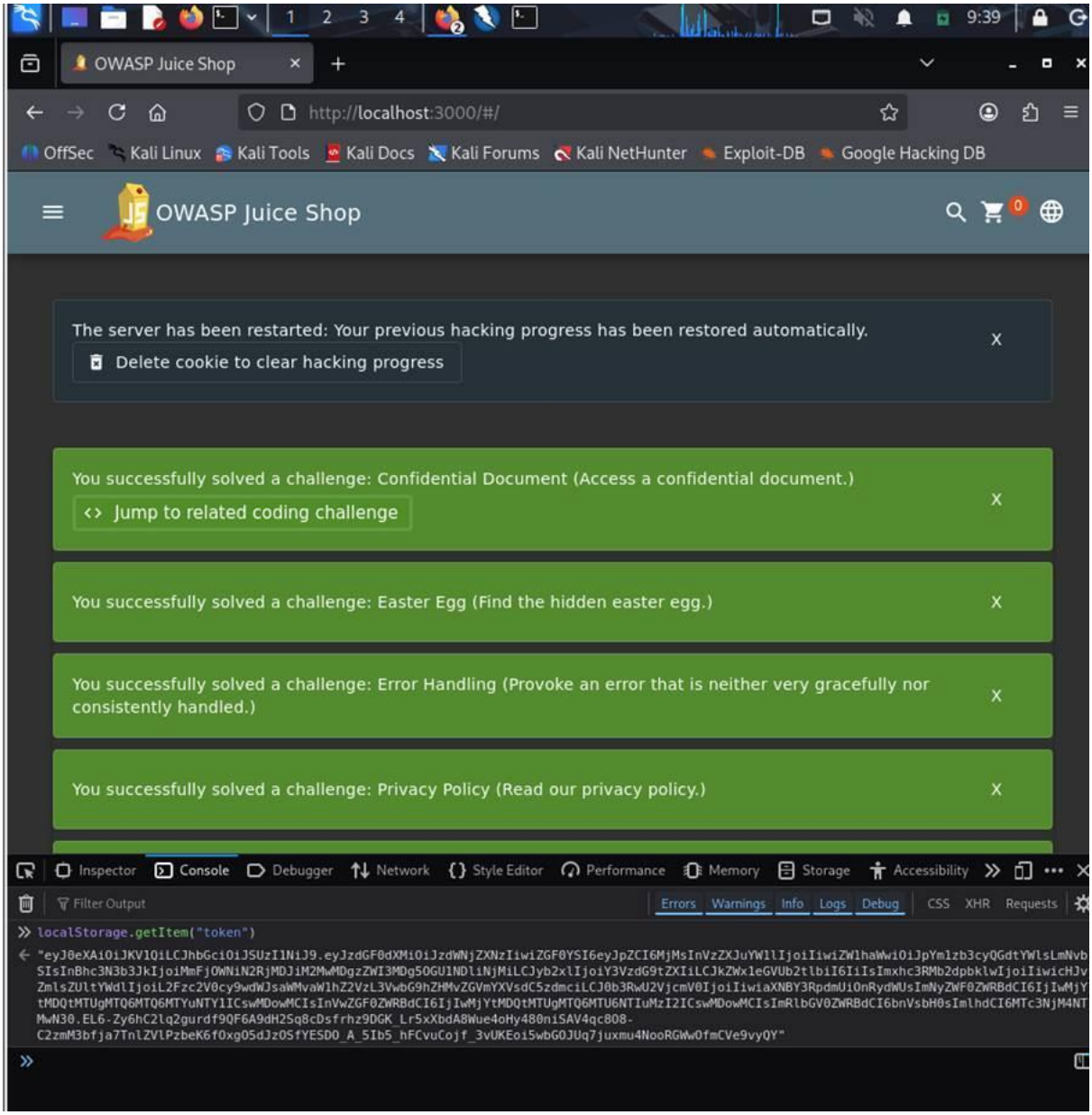
as that protection weakness client-side increases the value of an exposed token. A better approach will consist of storing those tokens in cookies HttpOnly and secure, which limit access via JavaScript and reduce the risk of client-side theft.





Manual Validation:

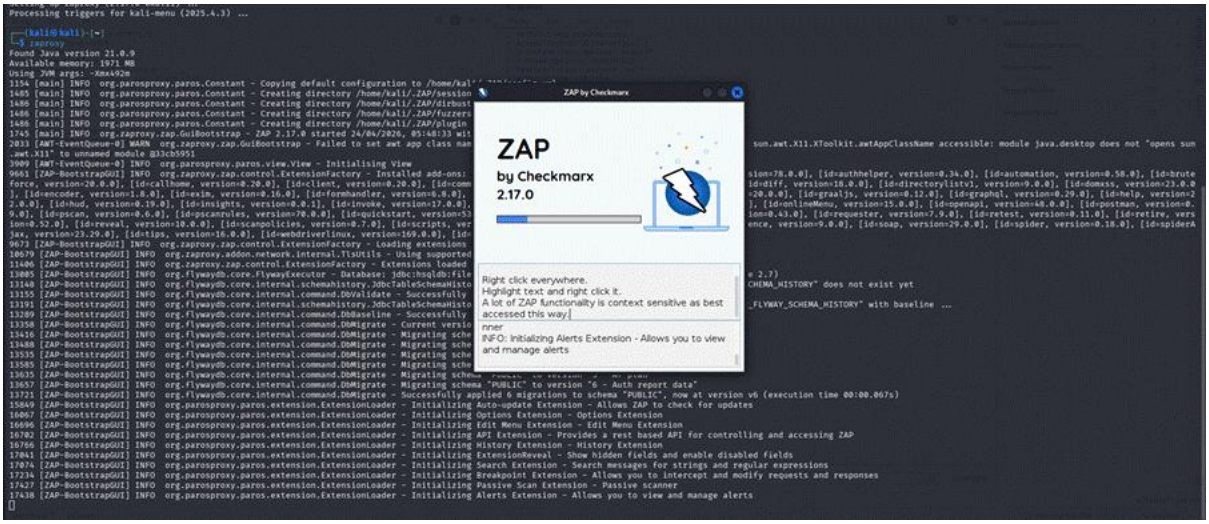
On the Juice Shop page, press F12 to open Developer Tools, then go to the Console tab. Type localStorage.getItem("token") and press Enter to display the stored JWT token

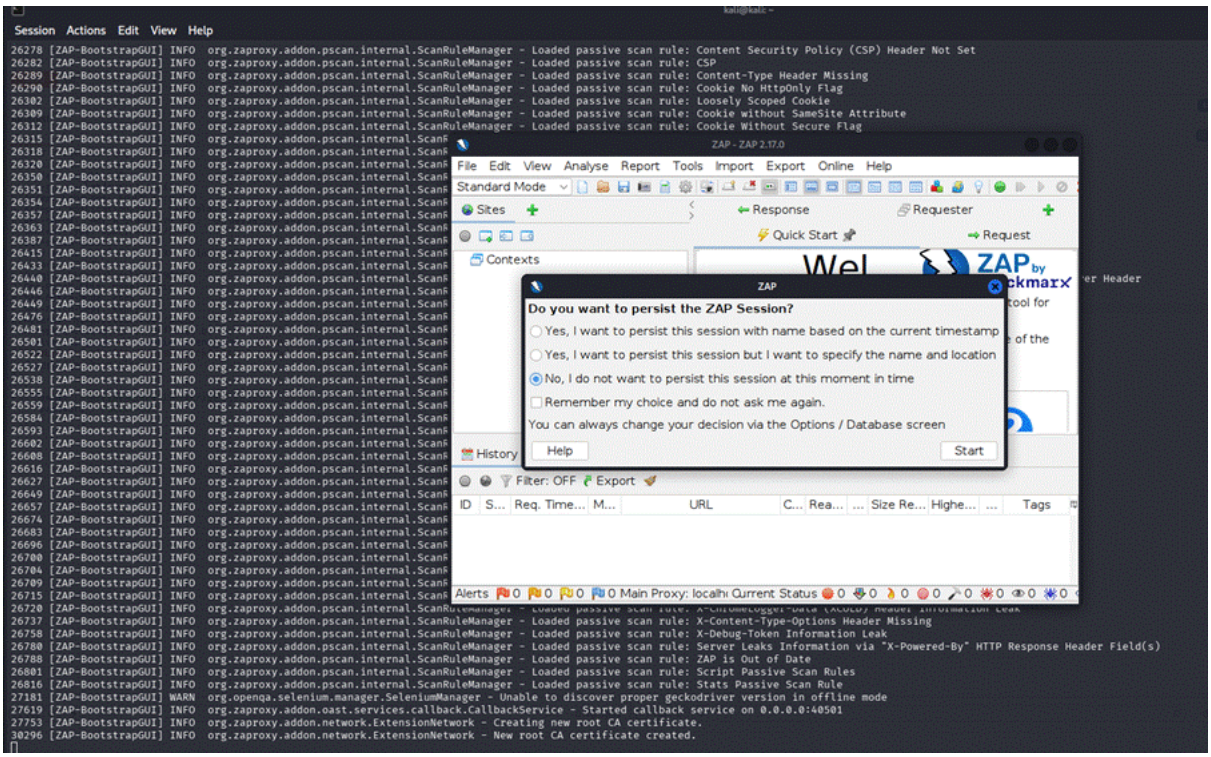


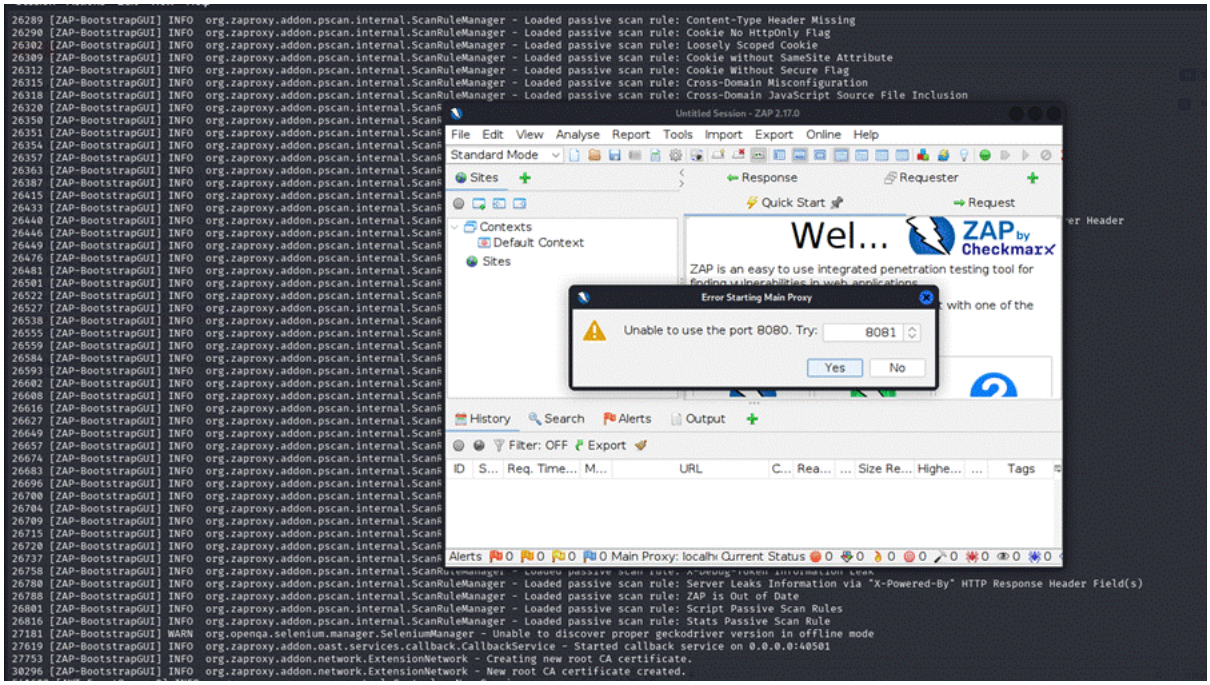
Finding 3 : Missing anti-clickjacking Header

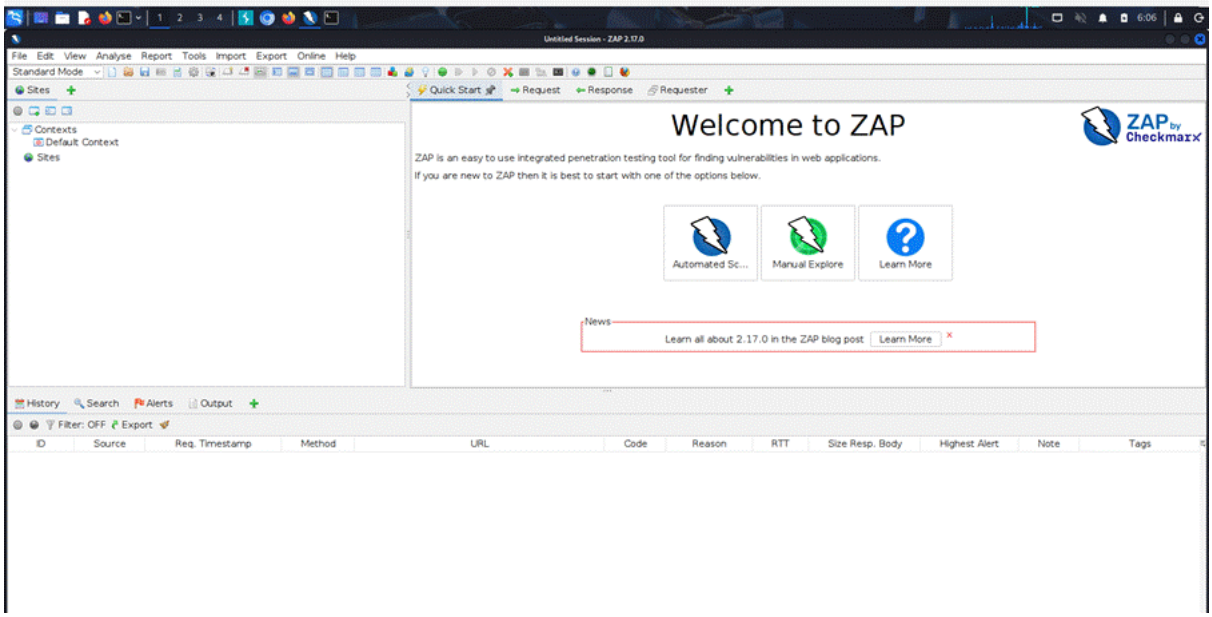
The analysis revealed also an inconsistent implementation of anti-clickjacking protections. While the main page correctly returned the X-Frame-Options: SAMEORIGIN header, certain specific endpoints, particularly some socket.io routes, did not return this header. A manual validation was carried out by comparing the headers of different routes in ZAP. This observation does not necessarily indicate an immediate critical vulnerability, especially since socket.io endpoints are less directly exposed to traditional clickjacking than full HTML pages, but it highlights an incomplete or inconsistent security configuration.

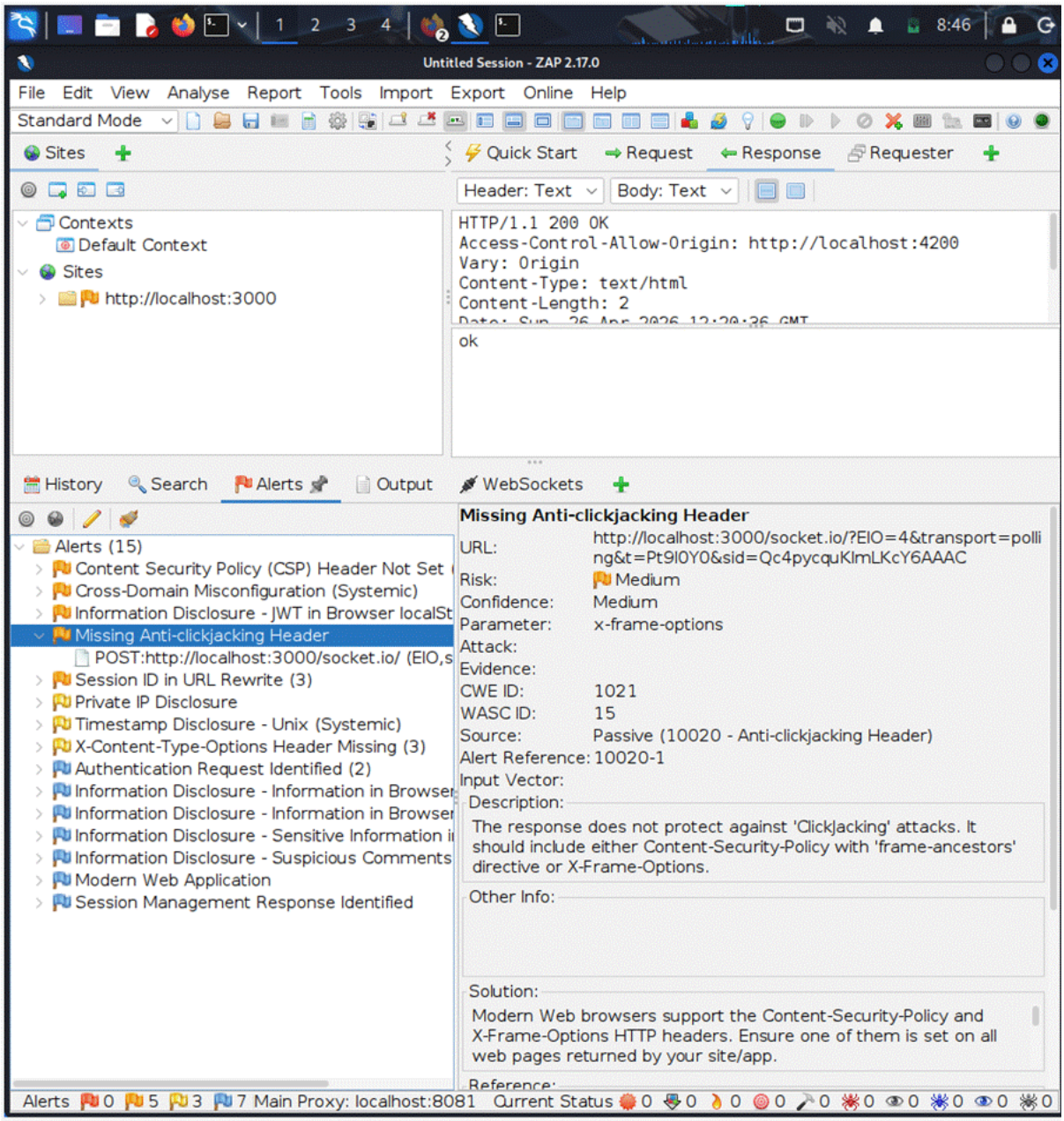
That inconsistence is important in term of defence, as it shows that some security protections are not uniformly applied to all the application components. In a real context, that type of irregularity can create blind spots or reveal insufficient governance of security policies. The recommended mitigation would be a global standardisation of security headers or the usage of CSP directives such as frame-ancestors to ensure consistent protection.

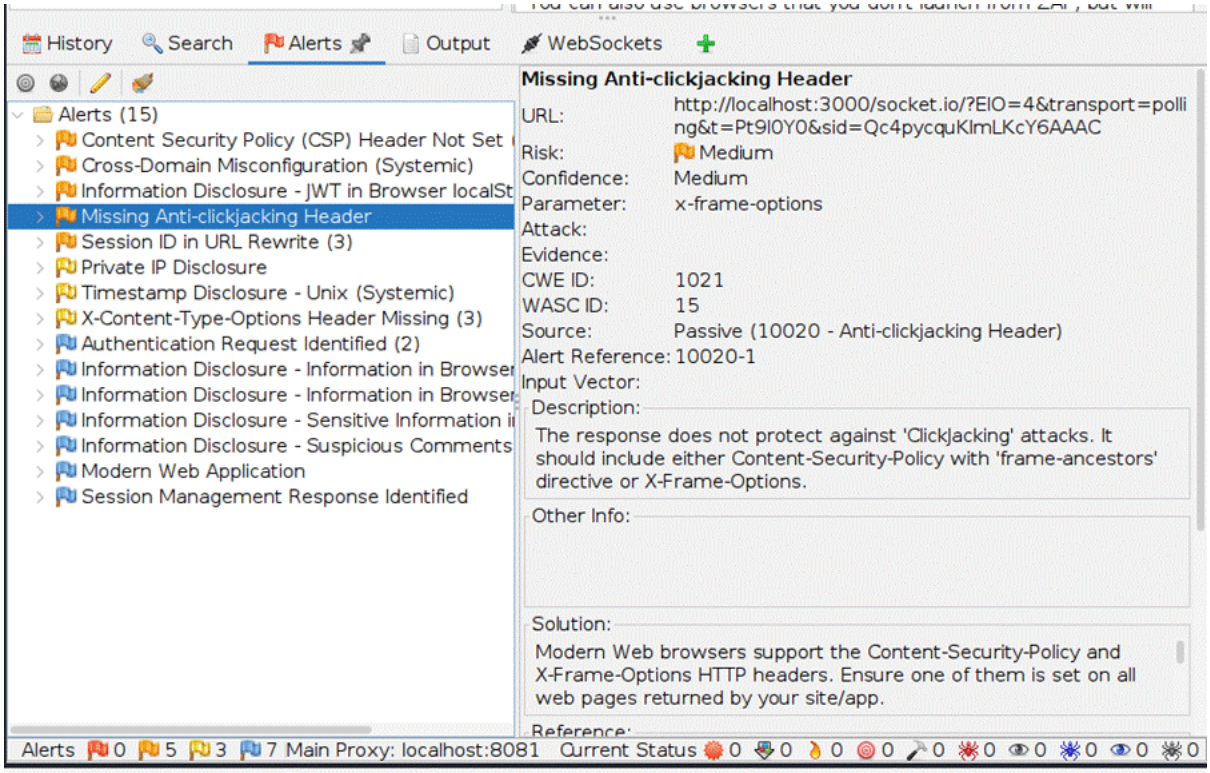


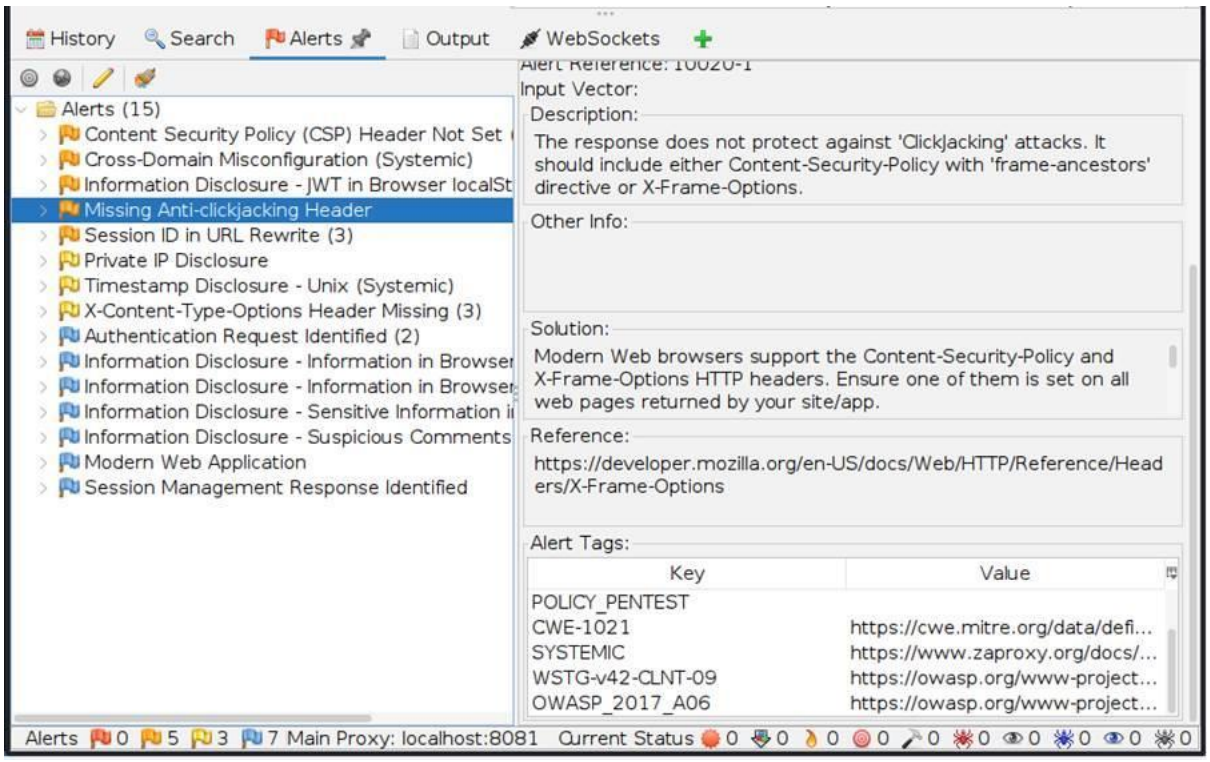



Untitled Session - ZAP 2.17.0
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Response
Requester
Quick Start
Request
Contexts
Default Context
Sites
Wel...
ZAP by Checkmarx
ZAP is an easy to use integrated penetration testing tool for
Error Starting Main Proxy
Unable to use the port 8080. Try: 8081
Yes
No
History
Search
Alerts
Output
Filter: OFF
Export
ID S... Req. Time... M... URL C... Rea... ... Size Re... Highe... ... Tags
Alerts 0 0 0 0 Main Proxy: localh Current Status 0 0 0 0 0 0 0 0 0


Untitled Session - ZAP 2.17.0
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Contexts
Default Context
Sites
Welcome to ZAP
ZAP by Checkmarx
ZAP is an easy to use integrated penetration testing tool for finding vulnerabilities in web applications.
If you are new to ZAP then it is best to start with one of the options below.
Automated Sc...
Manual Explore
Learn More
News
Learn all about 2.17.0 in the ZAP blog post
Learn More
History
Search
Alerts
Output
Filter: OFF
Export
ID Source Req. Timestamp Method URL Code Reason RTT Size Resp. Body Highest Alert Note Tags


Untitled Session - ZAP 2.17.0
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Header: Text
Body: Text
Contexts
Default Context
Sites
http://localhost:3000
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:4200
Vary: Origin
Content-Type: text/html
Content-Length: 2
ok
History
Search
Alerts
Output
WebSockets
Alerts (15)
Content Security Policy (CSP) Header Not Set
Cross-Domain Misconfiguration (Systemic)
Information Disclosure - JWT in Browser localSt
Missing Anti-clickjacking Header
POST:http://localhost:3000/socket.io/ (EIO,s
Session ID in URL Rewrite (3)
Private IP Disclosure
Timestamp Disclosure - Unix (Systemic)
X-Content-Type-Options Header Missing (3)
Authentication Request Identified (2)
Information Disclosure - Information in Browser
Information Disclosure - Information in Browser
Information Disclosure - Sensitive Information i
Information Disclosure - Suspicious Comments
Modern Web Application
Session Management Response Identified
Missing Anti-clickjacking Header
URL: http://localhost:3000/socket.io/?EIO=4&transport=polling&t=Pt9l0Y0&sid=Qc4pycquKlmLKcY6AAAC
Risk: Medium
Confidence: Medium
Parameter: x-frame-options
Attack:
Evidence:
CWE ID: 1021
WASC ID: 15
Source: Passive (10020 - Anti-clickjacking Header)
Alert Reference: 10020-1
Input Vector:
Description:
The response does not protect against 'Clickjacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
Other Info:
Solution:
Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.
Reference:
Alerts 0 5 3 7 Main Proxy: localhost:8081 Current Status 0 0 0 0 0 0 0 0 0 0





Manual Validation:

Step 1:

Scanner alert identified a missing header on the socket.io endpoint.

Step 2:
Manual inspection of response headers confirmed the absence of X-Frame-Options on this endpoint.
Step 3:
Comparison with the main page showed SAMEORIGIN was present elsewhere.
Result:
Inconsistent security header deployment.

Overall interpretation

DAST evaluation showed that results generated by OWASP ZAP should be interpreted critically rather than accepted automatically. Several low-value alerts, such as Timestamp Disclosure, Private IP Disclosure, or Suspicious Comments, were considered less relevant to the main analysis. The focus was therefore placed on findings that offered real analytical value: absence of CSP, JWT exposure, and inconsistencies in the implementation of security headers.

Overall, that analysis demonstrated that OWASP Juice Shop presents several real weaknesses more related to defense-in-depth security than to a single catastrophic vulnerability. The combination of an incomplete client-side hardening approach, exposed session mechanisms, and unevenly applied protections reflect issues frequently observed in real-world web applications. By combining ZAP's automated results with manual validation through header inspection, developer tools, and behavioral observation, the analysis provided a more precise, critical, and technically relevant view of the application's security posture.

References

[1] Lorem ipsum dolor
[2] sit amet, consectetuer
[3] adipiscing elit